

本节内容

单链表 定义

知识总览

线性表

逻辑结构

基本运算/操作

存储/物理结构

a_1

a_2

a_3

a_4

a_5

顺序表（顺序存储）

定义（如何用代码实现）

基本操作的实现

链表（链式存储）

单链表

定义（如何用代码实现）

基本操作的实现

双链表

循环链表

静态链表

知识总览

单链表的定义

什么是单链表

用代码定义一个单链表

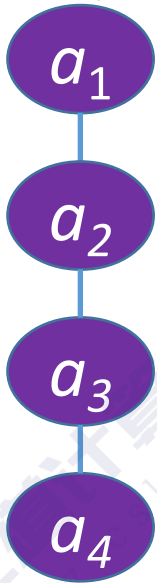
两种实现

带头结点

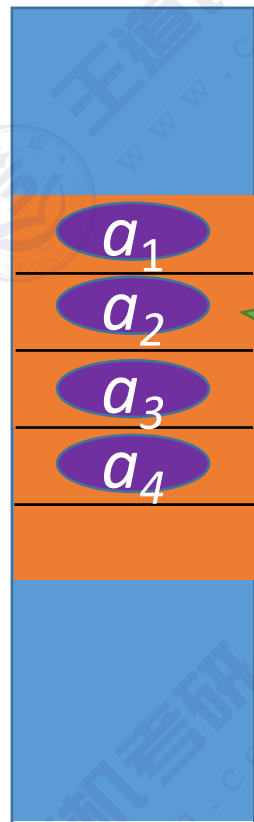
不带头结点

什么是单链表

逻辑结构：
线性表

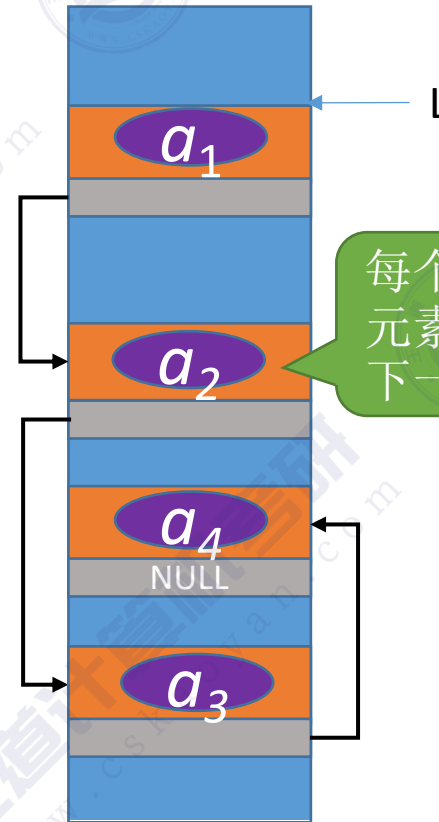


顺序表
(顺序存储)



每个结点中只
存放数据元素

单链表
(链式存储)



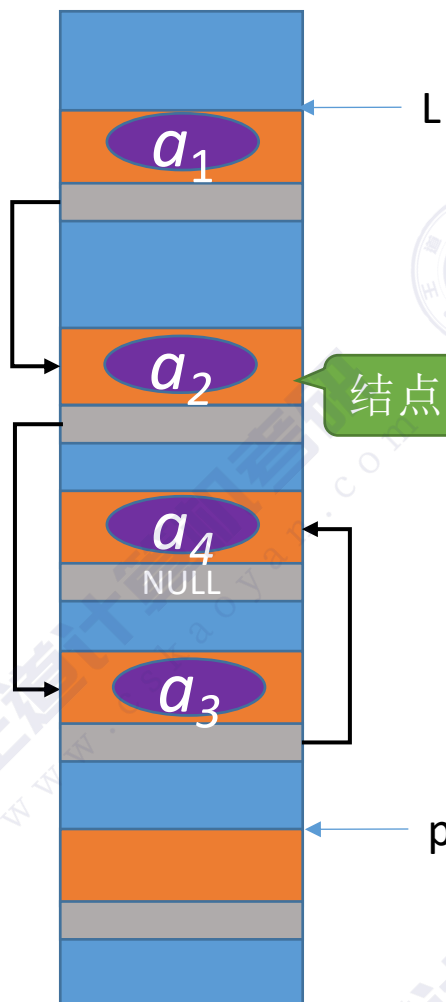
每个结点除了存放数据
元素外，还要存储指向
下一个节点的指针

优点：可随机存取，存储密度高
缺点：要求大片连续空间，改变容量不方便

优点：不要求大片连续空间，改变容量方便
缺点：不可随机存取，要耗费一定空间存放指针

用代码定义一个单链表

单链表
(链式存储)



结点

```
struct LNode{  
    ElemType data;  
    struct LNode *next;  
};
```

数据域

指针域

//定义单链表结点类型

//每个节点存放一个数据元素

//指针指向下一个节点

```
struct LNode * p = (struct LNode *) malloc(sizeof(struct LNode));
```



事情并不简单

增加一个新的结点：在内存中申请一个结点所需空间，并用指针 p 指向这个结点

typedef 关键字 —— 数据类型重命名

typedef <数据类型> <别名>

```
typedef int zhengshu;
```

```
typedef int *zhengshuzhizhen;
```

```
int x = 1;
```

```
int *p;
```

```
zhengshu x = 1;
```

```
zhengshuzhizhen p;
```

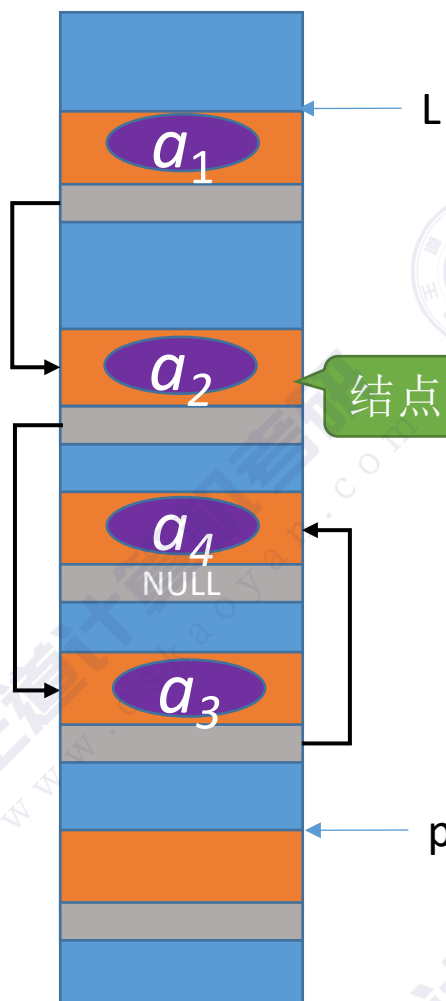
等价



巴啦啦能量

用代码定义一个单链表

单链表
(链式存储)



结点

```
struct LNode{  
    ElemType data;  
    struct LNode *next;  
};
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
struct LNode * p = (struct LNode *) malloc(sizeof(struct LNode));
```



增加一个新的结点：在内存中申请一个结点所需空间，并用指针 p 指向这个结点

typedef 关键字 —— 数据类型重命名

typedef <数据类型> <别名>

```
typedef struct LNode LNode;
```

```
LNode * p = (LNode *) malloc(sizeof(LNode));
```

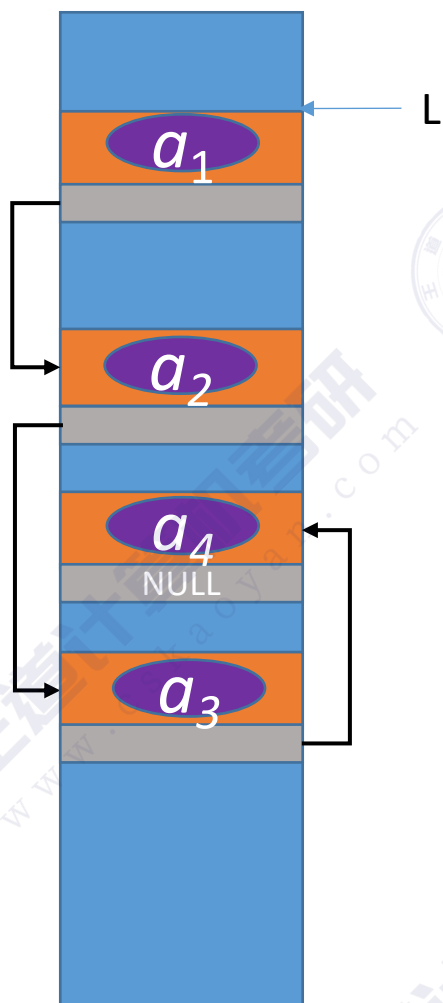
原来如此，简单!



巴啦啦能量

用代码定义一个单链表

单链表
(链式存储)



```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
} LNode, *LinkList;
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

LinkList
网络释义

```
struct LNode{  
    ElemType data;  
    struct LNode *next;  
};
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
typedef struct LNode LNode;  
typedef struct LNode *LinkList;
```

要表示一个单链表时，只需声明一个头指针 L，指向单链表的第一个结

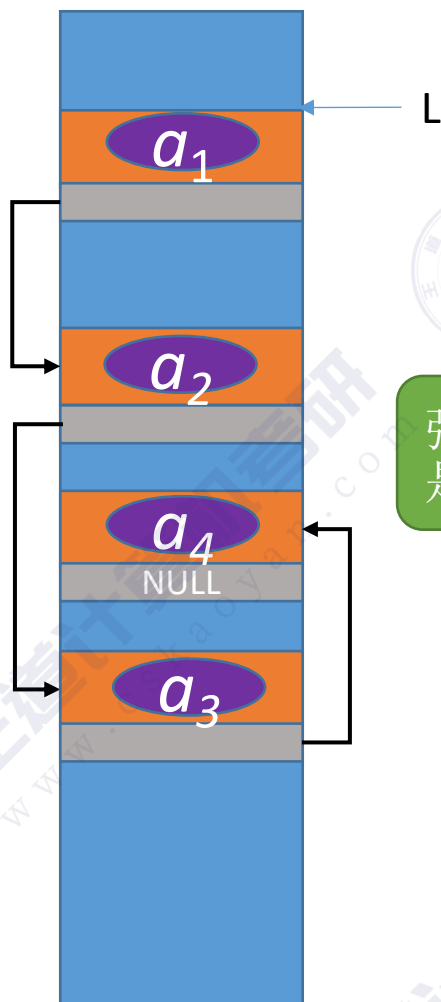
```
LNode * L; //声明一个指向单链表第一个结点的指针
```

或: LinkList L; //声明一个指向单链表第一个结点的指针

代码可读性更强

用代码定义一个单链表

单链表
(链式存储)



```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
}LNode, *LinkList;
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

```
LNode * GetElem(LinkList L, int i){
```

强调返回的
是一个结点

```
    int j=1;  
    LNode *p=L->next;  
    if(i==0)
```

强调这是一个
单链表

```
        return L;  
    if(i<1)  
        return NULL;  
    while(p!=NULL && j<i){  
        p=p->next;  
        j++;  
    }  
    return p;
```

强调这是一个单链表
强调这是一个结点

——使用 LinkList
——使用 LNode *

用代码定义一个单链表

头插法建立单链表的算法如下：

```
LinkedList List_HeadInsert(LinkedList &L) { // 逆向建立单链表
    LNode *s; int x;
    L = (LinkedList) malloc(sizeof(LNode)); // 创建头结点
    L->next = NULL; // 初始为空链表
    scanf("%d", &x); // 输入结点的值
    while(x != 9999) { // 输入 9999 表示结束
        s = (LNode*) malloc(sizeof(LNode)); // 创建新结点
        s->data = x;
        s->next = L->next;
        L->next = s; // 将新结点插入表中，L 为头指针
        scanf("%d", &x);
    }
    return L;
}
```

强调这是一个单链表
强调这是一个结点

——使用 LinkedList
——使用 LNode *

不带头结点的单链表

内存

```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
}LNode, *LinkList;
```

//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

//初始化一个空的单链表

```
bool InitList(LinkList &L) {
```

→ L = NULL; //空表, 暂时还没有任何结点

```
    return true;
```

```
}
```

```
void test(){
```

→ LinkList L; //声明一个指向单链表的指针

//初始化一个空表

→ InitList(L);

//.....后续代码.....

```
}
```

防止脏数据

注意, 此处
并没有创建
一个结点

//判断单链表是否为空

```
bool Empty(LinkList L) {
```

```
    if (L == NULL)
```

```
        return true;
```

```
    else
```

```
        return false;
```

```
}
```

或: bool Empty(LinkList L) {

```
    return (L==NULL);
```

```
}
```

头指针 L → NULL

带头结点的单链表

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;

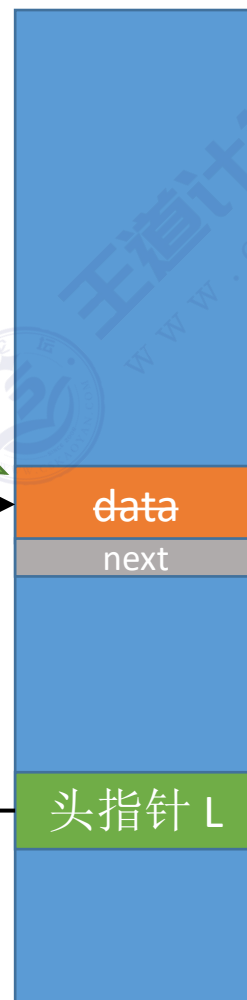
//初始化一个单链表 (带头结点)
bool InitList(LinkList &L) {
    → L = (LNode *) malloc(sizeof(LNode)); //分配一个头结点
    if (L==NULL) //内存不足, 分配失败
        return false;
    → L->next = NULL; //头结点之后暂时还没有节点
    return true;
}

void test(){
    → LinkList L; //声明一个指向单链表的指针
    //初始化一个空表
    → InitList(L);
    //.....后续代码.....
}
```

```
//判断单链表是否为空 (带头结点)
bool Empty(LinkList L) {
    if (L->next == NULL)
        return true;
    else
        return false;
}
```

内存

头结点不
存储数据



不带头结点 v.s. 带头结点

不带头

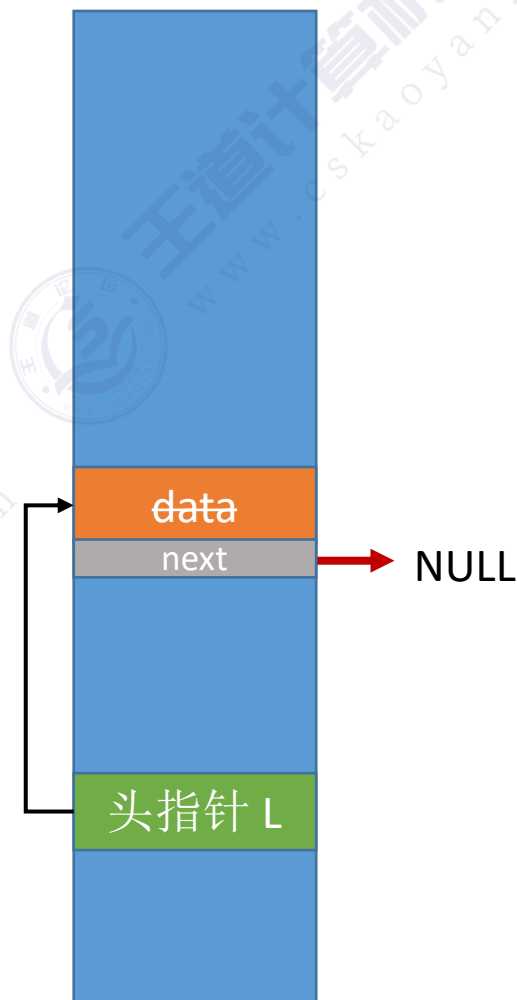


带头结点，写代码更方便，用过都说好



不带头结点，写代码更麻烦
对第一个数据结点和后续数据结点的处理需要用不同的代码逻辑
对空表和非空表的处理需要用不同的代码逻辑

带头



知识回顾与重要考点

单链表的定义

单链表

用“链式存储”(存储结构)实现了“线性结构”(逻辑结构)

一个结点存储一个数据元素

各结点间的先后关系用一个指针表示

用代码定义一个单链表

```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
}LNode, *LinkList;
```

两种实现

不带头结点

空表判断: $L == \text{NULL}$ 。写代码不方便

带头结点

空表判断: $L \rightarrow \text{next} == \text{NULL}$ 。写代码更方便

头结点不存数据,
只是为了操作方便

其他值得注意的点

typedef 关键字的用法

“LinkList” 等价于 “LNode *”
前者强调这是链表, 后者强调这是结点
合适的地方使用合适的名字, 代码可读性更高

本节内容

单链表

插入和删除

知识总览

单链表的插入删除

插入

按位序插入

带头结点

不带头结点

指定结点的后插操作

指定结点的前插操作

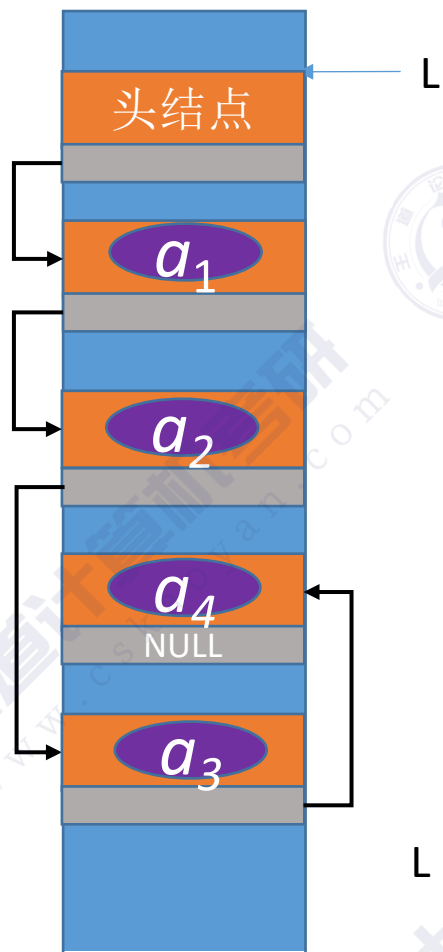
按位序删除

删除

指定结点的删除

关于简化图示的说明

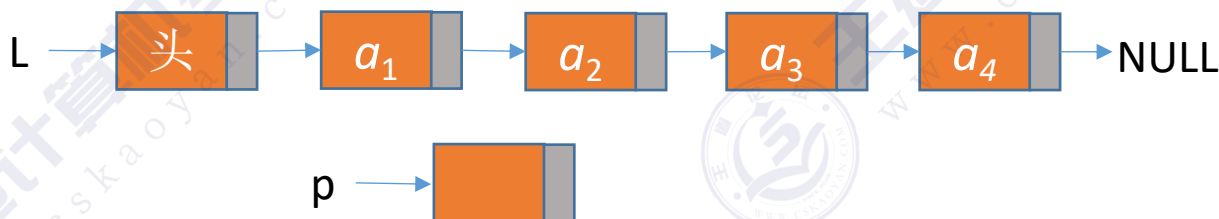
单链表
(带头结点)



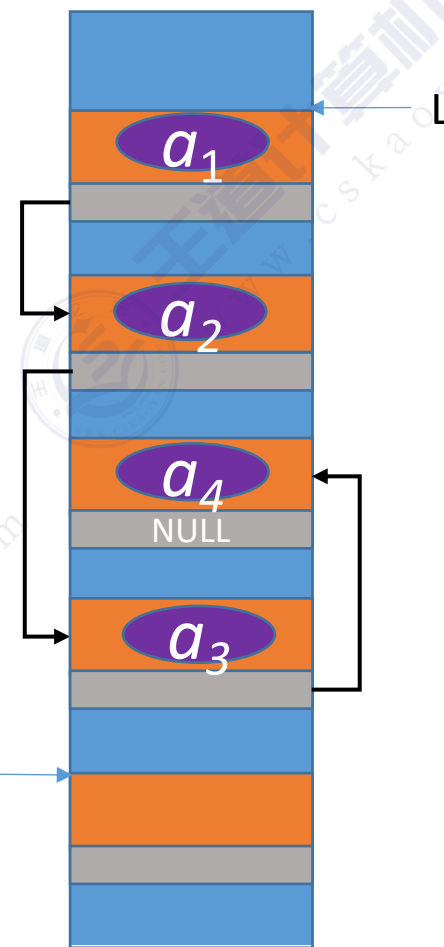
内存

```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
}LNode, *LinkList;
```

```
LNode * p = (LNode *) malloc(sizeof(LNode));  
free(p);
```



单链表
(不带头结点)



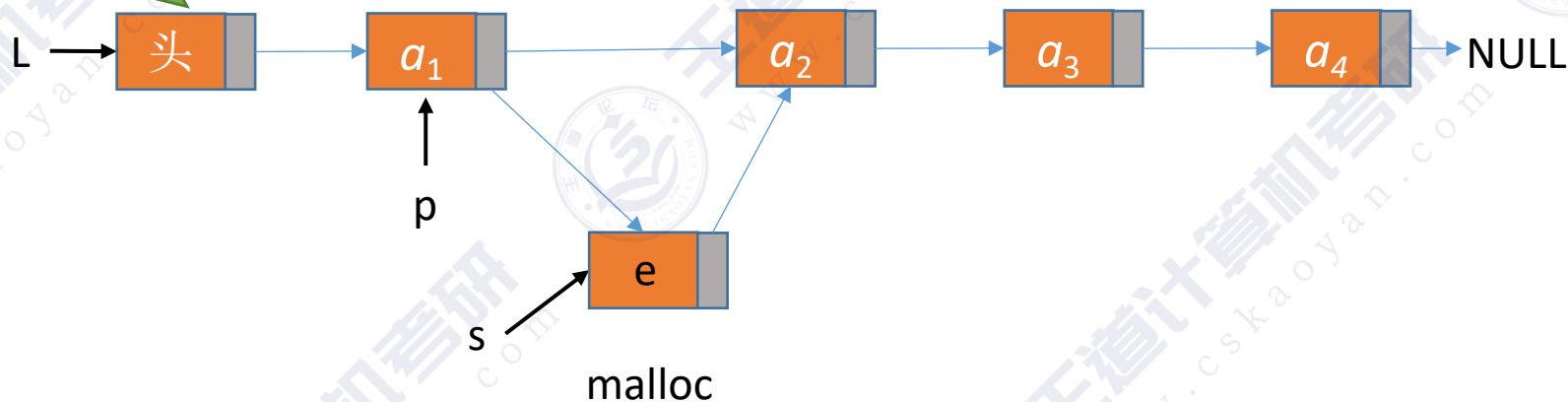
内存

按位序插入（带头结点）

ListInsert(&L,i,e): 插入操作。在表L中的第*i*个位置上插入指定元素e。

头结点可以看作
“第0个”结点

找到第 $i-1$ 个结点，
将新结点插入其后



按位序插入（带头结点）

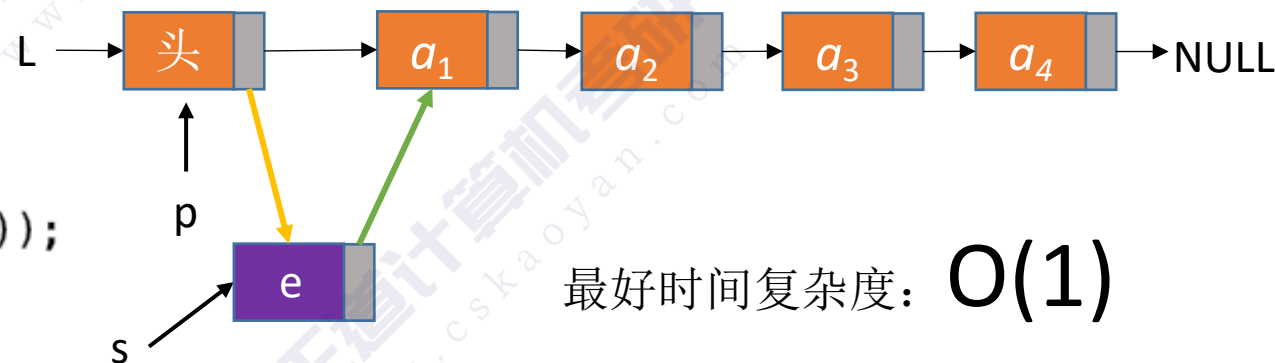
//在第 i 个位置插入元素 e （带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;       //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;    //将结点s连到p之后
    return true;  //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

①如果 $i = 1$ （插在表头）



最好时间复杂度: $O(1)$

注意：绿绿和黄黄顺序不能颠倒鸭！

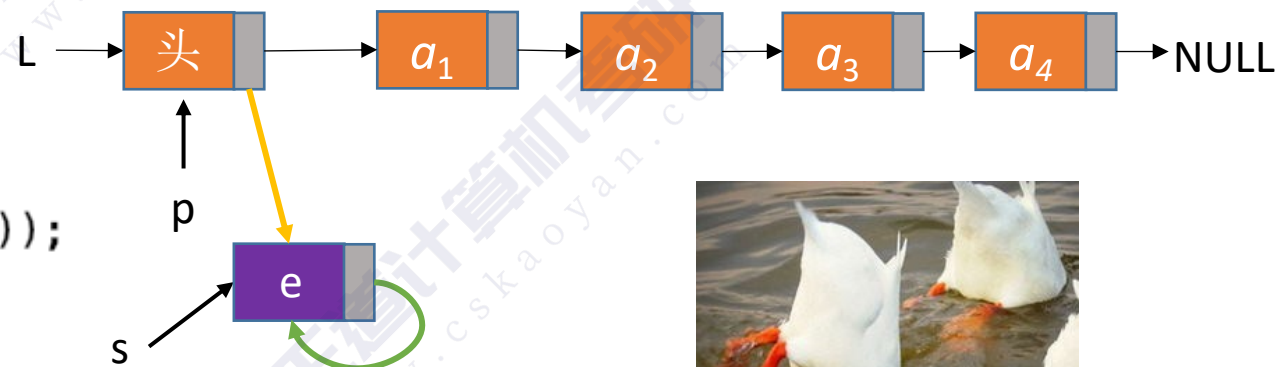
按位序插入（带头结点）

```
//在第 i 个位置插入元素 e（带头结点）
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;       //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;    //将结点s连到p之后
    return true;  //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

①如果 $i = 1$ （插在表头）



注意：绿绿和黄黄顺序不能颠倒鸭！

按位序插入（带头结点）

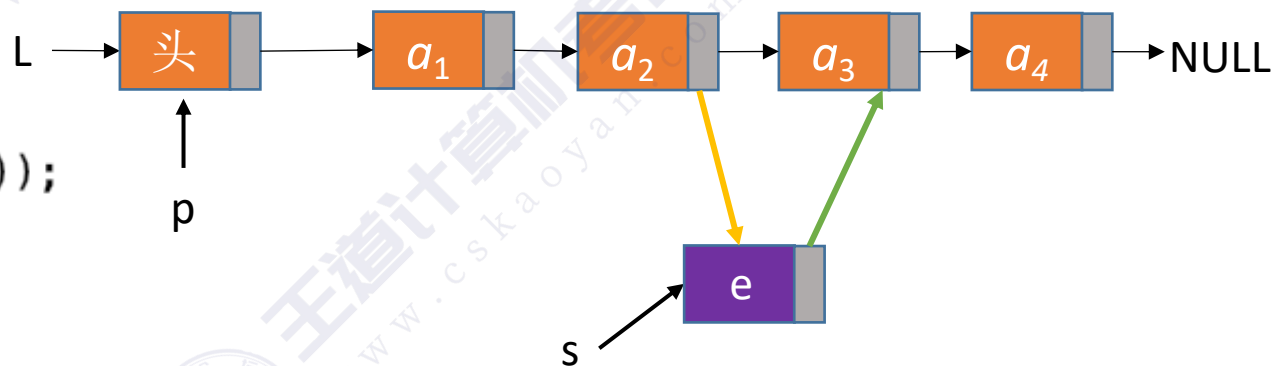
//在第 i 个位置插入元素 e （带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;        //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;    //将结点s连到p之后
    return true;  //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

②如果 $i = 3$ （插在表中）



按位序插入（带头结点）

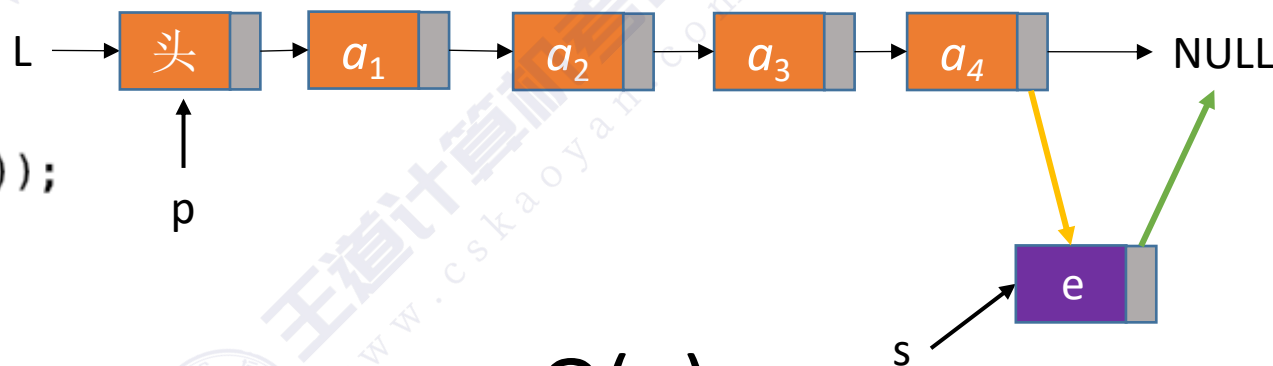
//在第 i 个位置插入元素 e （带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;        //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;    //将结点s连到p之后
    return true;  //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

③如果 $i = 5$ （插在表尾）



最坏时间复杂度: $O(n)$

按位序插入（带头结点）

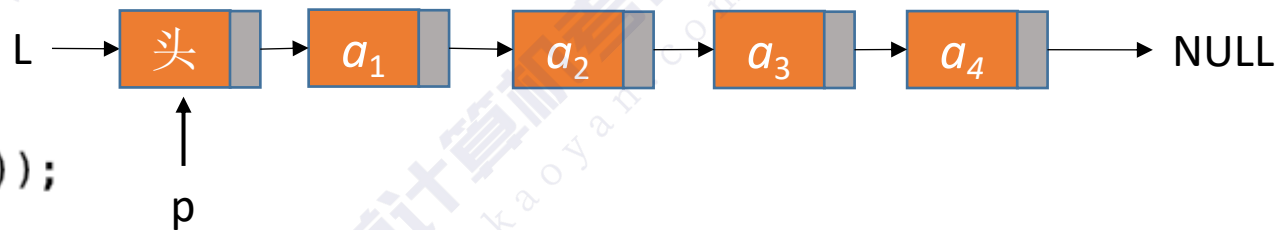
//在第 i 个位置插入元素 e （带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;        //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;      //将结点s连到p之后
    return true;    //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

④如果 $i = 6$ ($i > \text{Length}$)



按位序插入（带头结点）

//在第 i 个位置插入元素 e （带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;        //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;      //将结点s连到p之后
    return true;    //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

平均时间复杂度: $O(n)$

原来如此，简单！

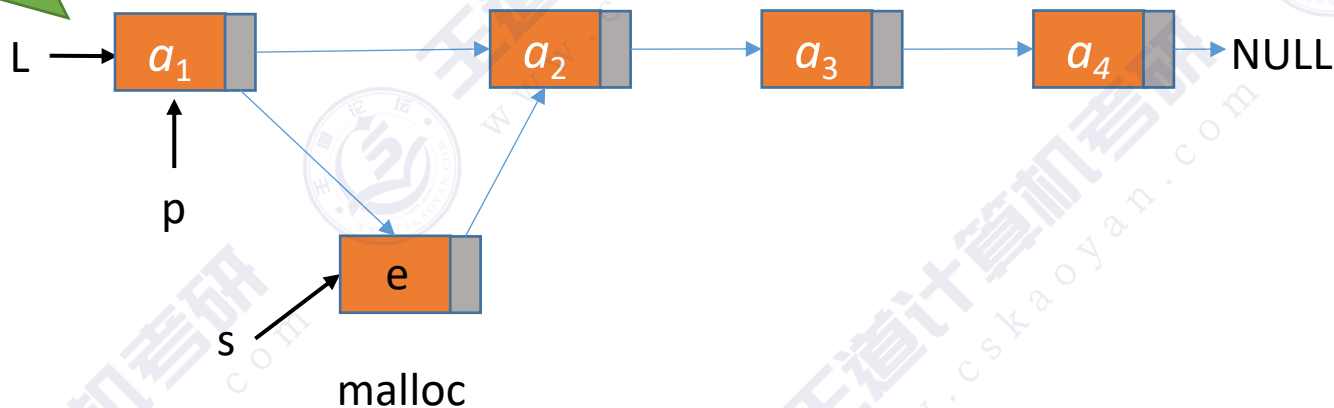


按位序插入（不带头结点）

ListInsert(&L,i,e): 插入操作。在表L中的第*i*个位置上插入指定元素e。

不存在“第0个”
结点，因此 $i=1$ 时
需要特殊处理

找到第 $i-1$ 个结点，
将新结点插入其后



按位序插入（不带头结点）

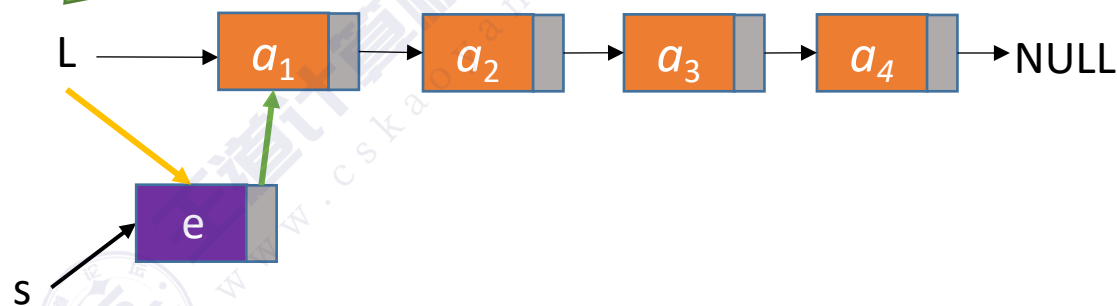
```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    if(i==1){ //插入第1个结点的操作与其他结点操作不同
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = e;
        s->next=L;
        L=s; //头指针指向新结点
        return true;
    }
    LNode *p; //指针p指向当前扫描到的结点
    int j=1; //当前p指向的是第几个结点
    p = L; //p指向第1个结点（注意：不是头结点）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;
    return true; //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

①如果 $i = 1$ （插在表头）

如果不带头结点，则插入、删除第1个元素时，需要更改头指针L



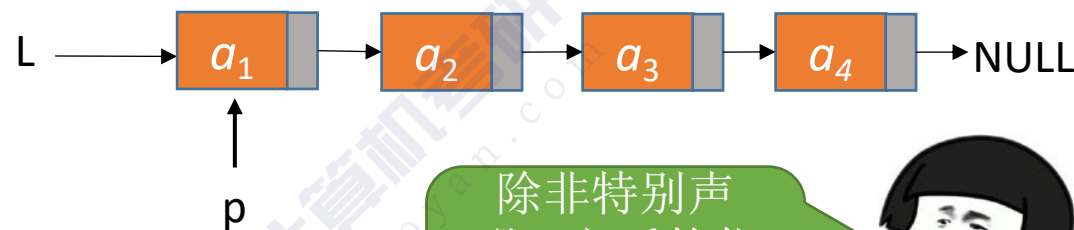
按位序插入（不带头结点）

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    if(i==1){    //插入第1个结点的操作与其他结点操作不同
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = e;
        s->next=L;
        L=s;    //头指针指向新结点
        return true;
    }
    LNode *p;    //指针p指向当前扫描到的结点
    int j=1;    //当前p指向的是第几个结点
    p = L;    //p指向第1个结点（注意：不是头结点）
    while (p!=NULL && j<i-1) {    //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL)    //i值不合法
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = e;
    s->next=p->next;
    p->next=s;
    return true;    //插入成功
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：

②如果 $i > 1$...



后续逻辑和带头结点的一样

除非特别声明，之后的代码默认带头结点



结论：不带头结点写代码更不方便，推荐用带头结点
注意：考试中带头、不带头都有可能考察，注意审题

指定结点的后插操作

//后插操作: 在 p 结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){  
    if (p==NULL)  
        return false;  
    LNode *s = (LNode *)malloc(sizeof(LNode));  
    if (s==NULL) //内存分配失败  
        return false;  
    s->data = e; //用结点s保存数据元素e  
    s->next = p->next;  
    p->next = s; //将结点s连到p之后  
    return true;  
}
```

```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
}LNode, *LinkList;
```

某些情况下有可能分配失败 (如内存不足)

神秘未知区域.....

x

y

.....神秘可知区域

p

s

e

时间复杂度: $O(1)$

指定结点的后插操作

//在第 i 个位置插入元素 e (带头结点)

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;        //L指向头结点, 头结点是第0个结点 (不存数据)
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
```

```
if(p==NULL)    //i值不合法
```

```
    return InsertNextNode(p, e);
```

```
LNode *s = (LNode *)malloc(sizeof(LNode));
```

```
s->data = e;
```

```
s->next=p->next;
```

```
p->next=s;    //将结点s连到p之后
```

```
return true;  //插入成功
```

```
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

//后插操作: 在 p 结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL)    //内存分配失败
        return false;
    s->data = e;    //用结点s保存数据元素e
    s->next=p->next;
    p->next=s;    //将结点s连到p之后
    return true;
```

```
}
```


指定结点的前插操作

//前插操作: 在 p 结点之前插入元素 e
bool InsertPriorNode (LNode * p , ElemType e)

如何找到 p 结点的前驱结点?



神秘未知区域.....

x

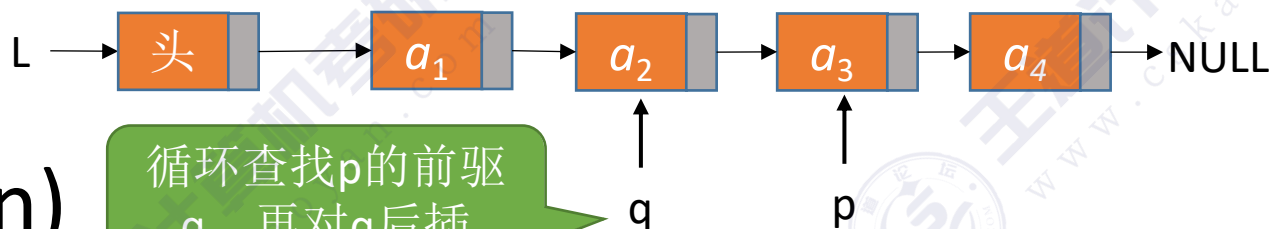
y

.....神秘可知区域

p

传入头指针

//前插操作: 在 p 结点之前插入元素 e
bool InsertPriorNode (LinkList L , LNode * p , ElemType e)



高清无码
拒绝神秘

时间复杂度: $O(n)$

循环查找 p 的前驱
 q , 再对 q 后插

指定结点的前插操作

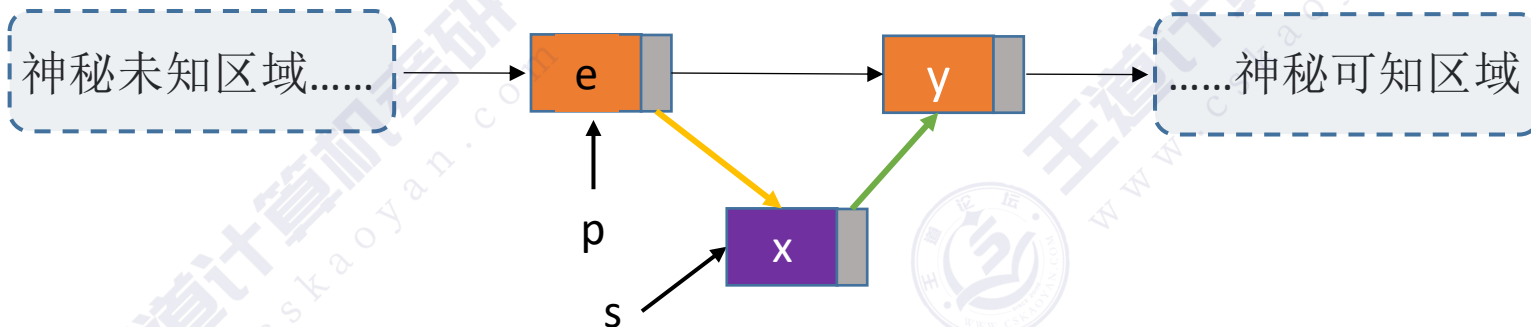
//前插操作：在 p 结点之前插入元素 e

```
bool InsertPriorNode (LNode *p, ElemType e){  
    if (p==NULL)  
        return false;  
    LNode *s = (LNode *)malloc(sizeof(LNode));  
    if (s==NULL) //内存分配失败  
        return false;  
    s->next=p->next;  
    p->next=s;  
    s->data=p->data;  
    p->data=e;  
    return true;  
}
```

//新结点 s 连到 p 之后
//将 p 中元素复制到 s 中
// p 中元素覆盖为 e



时间复杂度： $O(1)$



指定结点的前插操作

//前插操作：在 p 结点之前插入结点 s

```
bool InsertPriorNode (LNode *p, LNode *s){  
    if (p==NULL || s==NULL)  
        return false;  
    s->next=p->next;  
    p->next=s;  
    ElemType temp=p->data; //交换数据域部分  
    p->data=s->data;  
    s->data=temp;  
    return true;  
}
```

王道书版本

神秘未知区域.....

x

p

s

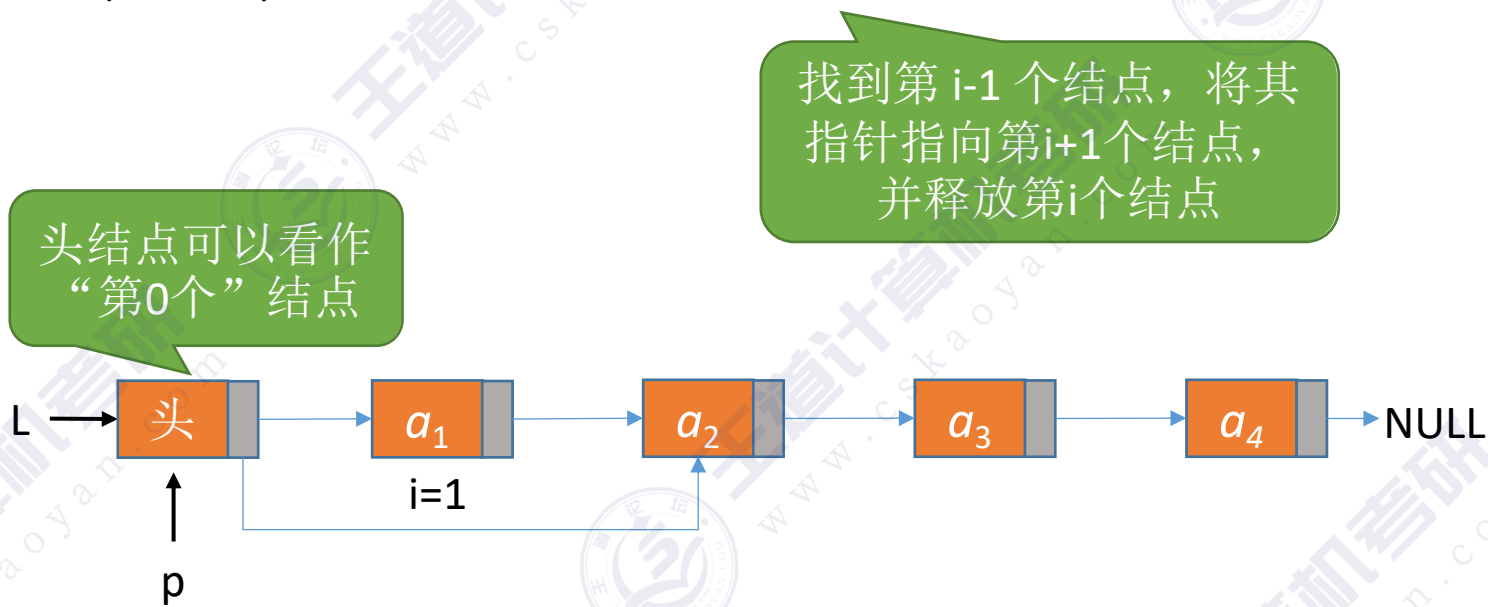
e

y

.....神秘可知区域

按位序删除（带头结点）

ListDelete(&L,i,&e): 删除操作。删除表L中第*i*个位置的元素，并用e返回删除元素的值。

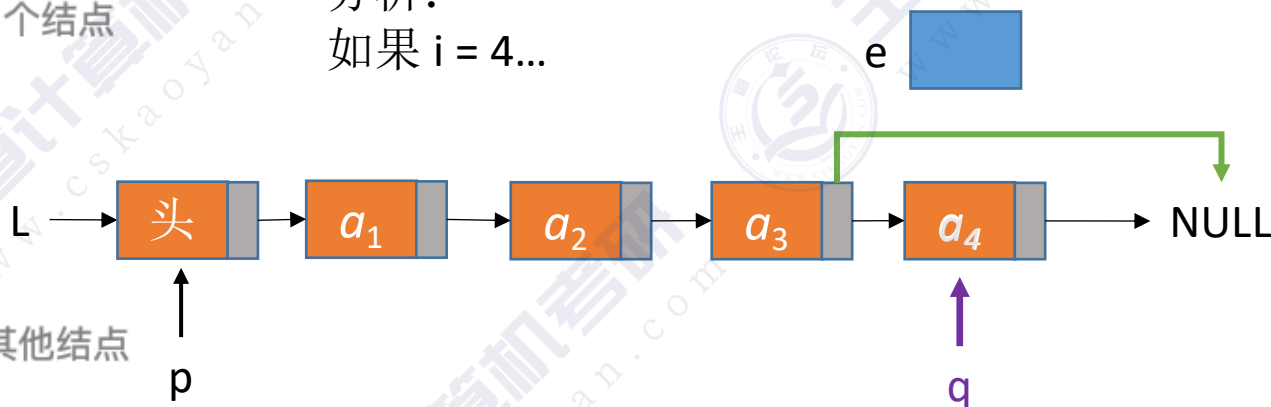


按位序删除（带头结点）

```
bool ListDelete(LinkList &L, int i, ElemType &e){
    if(i<1)
        return false;
    LNode *p;    //指针p指向当前扫描到的结点
    int j=0;      //当前p指向的是第几个结点
    p = L;       //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第 i-1 个结点
        p=p->next;
        j++;
    }
    if(p==NULL) //i值不合法
        return false;
    if(p->next == NULL) //第i-1个结点之后已无其他结点
        return false;
    LNode *q=p->next; //令q指向被删除结点
    e = q->data;       //用e返回元素的值
    p->next=q->next;   //将*q结点从链中“断开”
    free(q);          //释放结点的存储空间
    return true;
}
```

```
typedef struct LNode{
    ElemType data;
    struct LNode *next;
}LNode, *LinkList;
```

分析：
如果 $i = 4...$



最坏、平均时间复杂度： $O(n)$

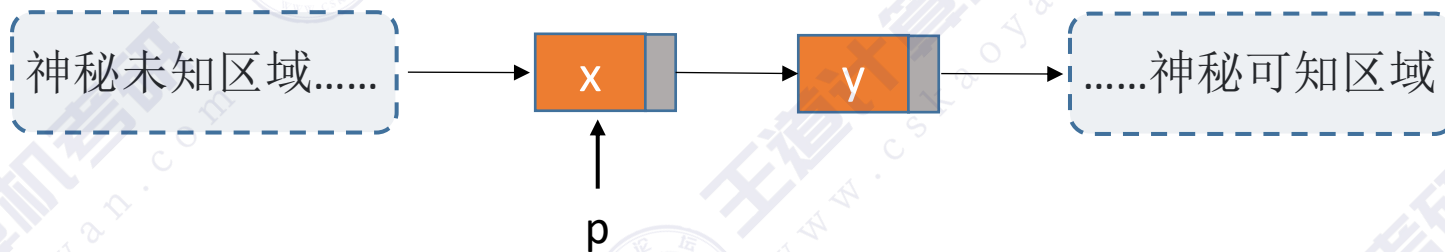
最好时间复杂度： $O(1)$

如果不带头结点，删除第1个元素，是否需要特殊处理？



指定结点的删除

```
//删除指定结点 p  
bool DeleteNode (LNode *p)
```



删除结点 p ，需要修改其前驱结点的 `next` 指针

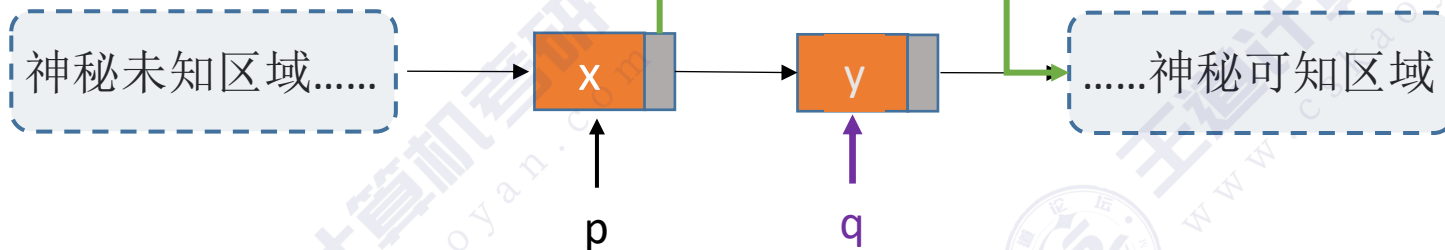
方法1：传入头指针，循环寻找 p 的前驱结点
方法2：偷天换日（类似于结点前插的实现）

指定结点的删除

//删除指定结点 p

```
bool DeleteNode (LNode *p){  
    if (p==NULL)  
        return false;  
    LNode *q=p->next;  
    p->data=p->next->data;  
    p->next=q->next;  
    free(q);  
    return true;  
}
```

//令 q 指向 $*p$ 的后继结点
//和后继结点交换数据域
//将 $*q$ 结点从链中“断开”
//释放后继结点的存储空间



可能是指向一个结点，
也可能是指向NULL

时间复杂度: $O(1)$

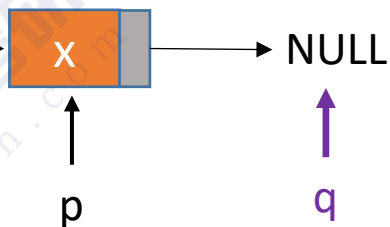
指定结点的删除

//删除指定结点 p

```
bool DeleteNode (LNode *p){  
    if (p==NULL)  
        return false;  
    LNode *q=p->next;  
    p->data=p->next->data;  
    p->next=q->next;  
    free(q);  
    return true;  
}
```

//令 q 指向 $*p$ 的后继结点
//和后继结点交换数据域
//将 $*q$ 结点从链中“断开”
//释放后继结点的存储空间

神秘未知区域.....



单链表的局限性:
无法逆向检索, 有
时候不太方便



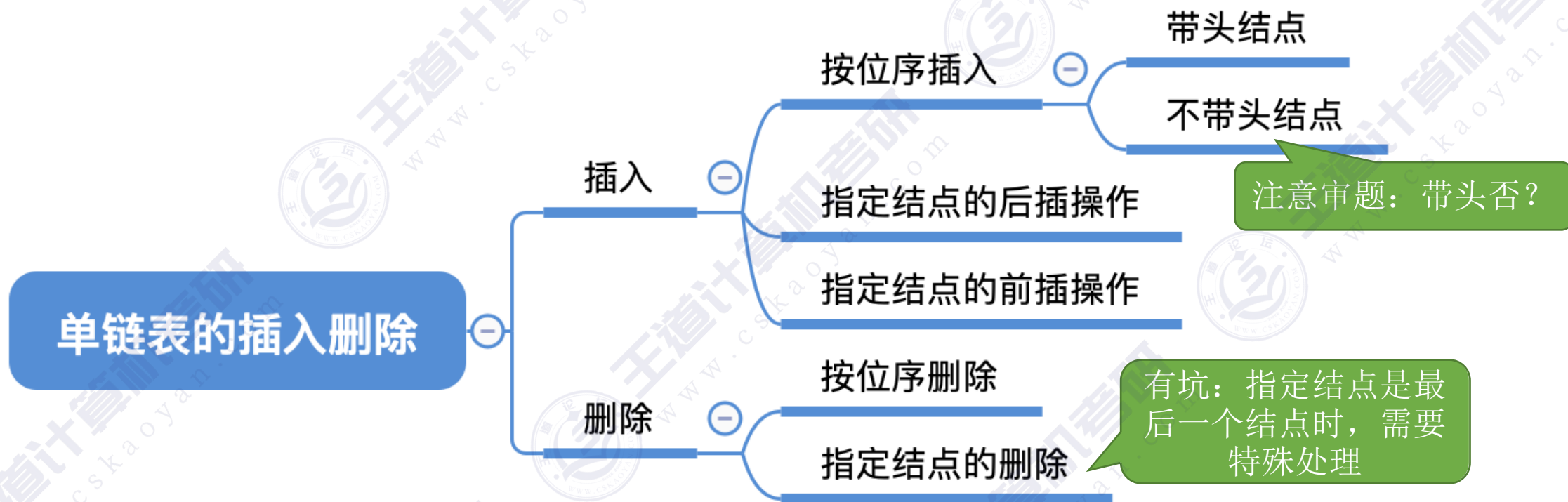
如果 p 是最后
一个结点...

只能从表头开始依
次寻找 p 的前驱,
时间复杂度 $O(n)$



思考考

知识回顾与重要考点



Tips:

1. 这些代码都要会写，都重要
2. 打牢基础，慢慢加速
3. 体会带头结点、不带头结点的代码区别
4. 体会“封装”的好处

封装的好处

小功能模块化，
代码逻辑清晰

//在第 i 个位置插入元素 e (带头结点)

```
bool ListInsert(LinkList &L, int i, ElemType e){
```

```
    if(i<1)
```

```
        return false;
```

```
    LNode *p;    //指针 $p$ 指向当前扫描到的结点
```

```
    int j=0;    //当前 $p$ 指向的是第几个结点
```

```
    p = L;    //L指向头结点，头结点是第0个结点 (不存数据)
```

```
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
```

```
        p=p->next;
```

```
        j++;
```

```
    }
```

```
    if(p==NULL)    //i值不合法
```

```
        return InsertNextNode(p, e);
```

```
    LNode *s = (LNode *)malloc(sizeof(LNode));
```

```
    s->data = e;
```

```
    s->next=p->next;
```

```
    p->next=s;    //将结点 $s$ 连到 $p$ 之后
```

```
    return true;    //插入成功
```

```
}
```

也可以称为“基本操作”。对
书本的概念理解不要教条化

//后插操作：在 p 结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){
```

```
    if (p==NULL)
```

```
        return false;
```

```
    LNode *s = (LNode *)malloc(sizeof(LNode));
```

```
    if (s==NULL)    //内存分配失败
```

```
        return false;
```

```
    s->data = e;    //用结点 $s$ 保存数据元素 $e$ 
```

```
    s->next=p->next;
```

```
    p->next=s;    //将结点 $s$ 连到 $p$ 之后
```

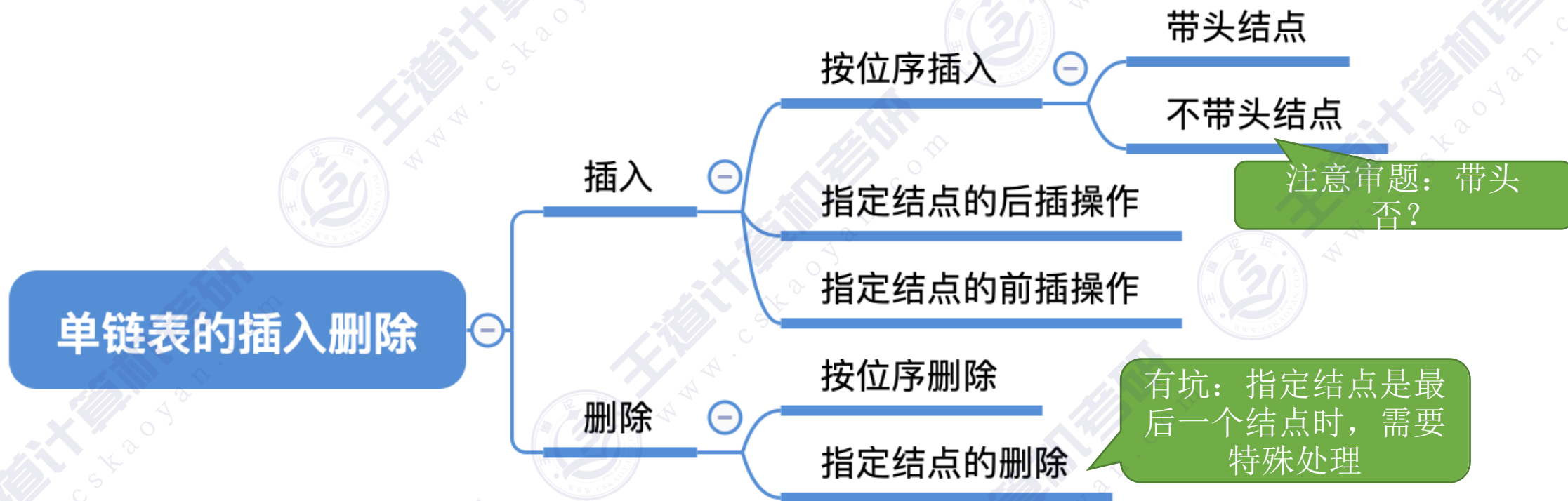
```
    return true;
```

```
}
```

指针 p 指向第 $i-1$ 个结点

p 后插入新元素 e

知识回顾与重要考点



Tips:

1. 这些代码都要会写，都重要
2. 打牢基础，慢慢加速
3. 体会带头结点、不带头结点的代码区别
4. 体会“封装”的好处

本节内容

单链表 查找

知识总览

单链表的查找

按位查找

按值查找

注：本节只探讨“带头结点”的情况

GetElem(L,i): **按位查找**操作。获取表L中第i个位置的元素的值。

LocateElem(L,e): **按值查找**操作。在表L中查找具有给定关键字值的元素。

按位查找

//在第 i 个位置插入元素 e (带头结点)

```
bool ListInsert(LinkList &L, int i, ElemType e){
```

```
    if(i<1)
```

```
        return false;
```

```
    LNode *p;    //指针p指向当前扫描到的结点
```

```
    int j=0;    //当前p指向的是第几个结点
```

```
    p = L;    //L指向头结点, 头结点是第0个结点 (不存数据)
```

```
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
```

```
        p=p->next;
```

```
        j++;
```

```
    }
```

```
    if(p==NULL) //i值不合法
```

```
        return false;
```

```
    LNode *s = (LNode *)malloc(sizeof(LNode));
```

```
    s->data = e;
```

```
    s->next=p->next;
```

```
    p->next=s;    //将结点s连到p之后
```

```
    return true;    //插入成功
```

```
}
```

```
bool ListDelete(LinkList &L, int i, ElemType &e){
```

```
    if(i<1)
```

```
        return false;
```

```
    LNode *p;    //指针p指向当前扫描到的结点
```

```
    int j=0;    //当前p指向的是第几个结点
```

```
    p = L;    //L指向头结点, 头结点是第0个结点 (不存数据)
```

```
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
```

```
        p=p->next;
```

```
        j++;
```

```
    }
```

```
    if(p==NULL) //i值不合法
```

```
        return false;
```

```
    if(p->next == NULL) //第  $i-1$  个结点之后已无其他结点
```

```
        return false;
```

```
    LNode *q=p->next;    //令q指向被删除结点
```

```
    e = q->data;    //用e返回元素的值
```

```
    p->next=q->next;    //将*q结点从链中“断开”
```

```
    free(q);    //释放结点的存储空间
```

```
    return true;    //删除成功
```

```
}
```

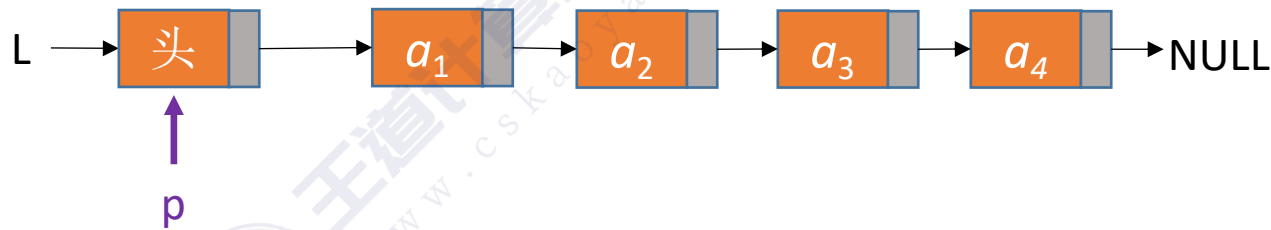
按位查找

//按位查找, 返回第 i 个元素 (带头结点)

```
ListNode * GetElem(LinkList L, int i){  
    if(i<0)  
        return NULL;  
    ListNode *p;    //指针p指向当前扫描到的结点  
    int j=0;        //当前p指向的是第几个结点  
    p = L;          //L指向头结点, 头结点是第0个结点 (不存数据)  
    while (p!=NULL && j<i) {    //循环找到第 i 个结点  
        p=p->next;  
        j++;  
    }  
    return p;  
}
```

边界情况:

①如果 $i = 0$



按位查找



//按位查找, 返回第 i 个元素 (带头结点)

```
LNode * GetElem(LinkList L, int i){
```

```
    if(i<0)
```

```
        return NULL;
```

```
    LNode *p;    //指针p指向当前扫描到的结点
```

```
    int j=0;    //当前p指向的是第几个结点
```

```
    ➡ p = L;    //L指向头结点, 头结点是第0个结点 (不存数据)
```

```
    while (p!=NULL && j<i) { //循环找到第  $i$  个结点
```

```
    ➡    p=p->next;
```

```
        j++;
```

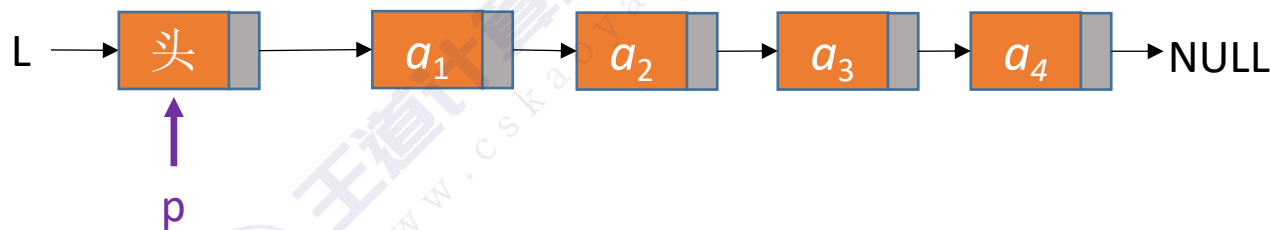
```
    }
```

```
    return p;
```

```
}
```

边界情况:

②如果 $i = 8$



按位查找

//按位查找, 返回第 i 个元素 (带头结点)

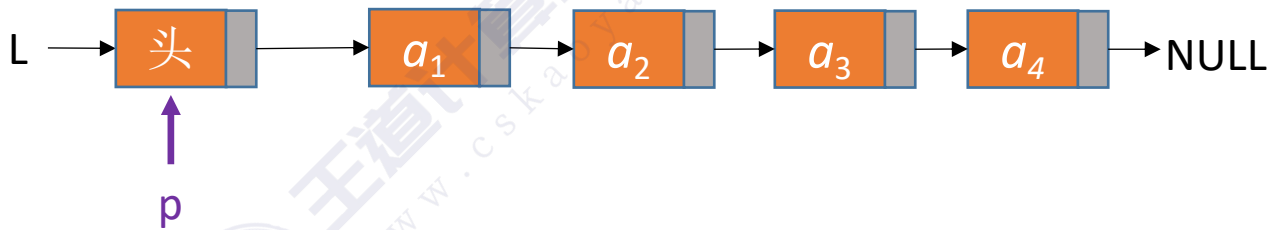
```
ListNode * GetElem(LinkList L, int i){  
    if(i<0)  
        return NULL;  
    ListNode *p;    //指针p指向当前扫描到的结点  
    int j=0;        //当前p指向的是第几个结点  
    p = L;          //L指向头结点, 头结点是第0个结点 (不存数据)  
    while (p!=NULL && j<i) { //循环找到第  $i$  个结点  
        p=p->next;  
        j++;  
    }  
    return p;  
}
```

普通情况:

③如果 $i = 3$



脑补一下咯



平均时间复杂度: $O(n)$

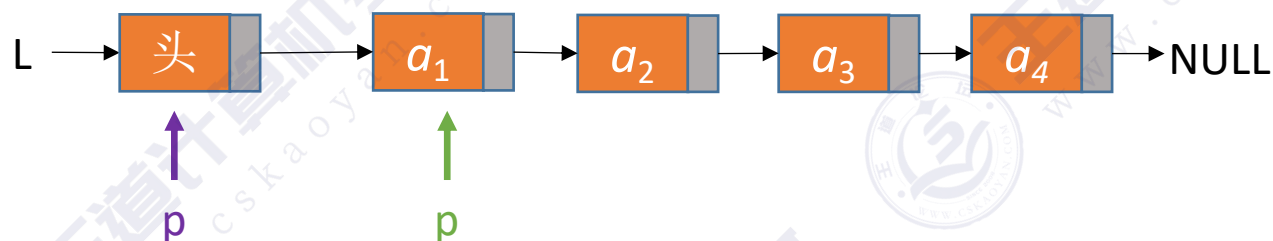
按位查找

王道书
版本

//按位查找, 返回第 i 个元素 (带头结点)

```
LNode * GetElem(LinkList L, int i){  
    if(i<0)  
        return NULL;  
    LNode *p;    //指针p指向当前扫描到的结点  
    int j=0;      //当前p指向的是第几个结点  
    p = L;        //L指向头结点, 头结点是第0个结点 (不存数据)  
    while (p!=NULL && j<i) {    //循环找到第 i 个结点  
        p=p->next;  
        j++;  
    }  
    return p;  
}
```

```
LNode * GetElem(LinkList L, int i){  
    int j=1;  
    LNode *p=L->next;  
    if(i==0)  
        return L;  
    if(i<1)  
        return NULL;  
    while(p!=NULL && j<i){  
        p=p->next;  
        j++;  
    }  
    return p;  
}
```



封装（基本操作）的好处

避免重复代码，
简洁、易维护

//在第 i 个位置插入元素 e (带头结点)

```
bool ListInsert(LinkList &L, int i, ElemType e){
    if(i<1)
        return false;
```

```
    LNode *p = GetElem(L, i-1); //找到第 $i-1$ 个结点
```

```
    int j=0; //当前 $p$ 指向的是第 $j$ 个结点
    p = L; //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
```

```
    if(p==NULL) //i值不合法
```

```
        return false;
    return InsertNextNode(p, e);
```

```
    s->data = e;
    s->next=p->next;
    p->next=s; //将结点 $s$ 连到 $p$ 之后
    return true; //插入成功
```

p后插入新元素e

```
bool ListDelete(LinkList &L, int i, ElemType &e){
    if(i<1)
        return false;
```

```
    LNode *p = GetElem(L, i-1); //找到第 $i-1$ 个结点
```

```
    p = L; //L指向头结点，头结点是第0个结点（不存数据）
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点
        p=p->next;
        j++;
    }
```

//后插操作：在 p 结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){
```

```
    if (p==NULL)
        return false;
```

```
    LNode *s = (LNode *)malloc(sizeof(LNode));
```

```
    if (s==NULL) //内存分配失败
```

```
        return false;
```

```
    s->data = e; //用结点 $s$ 保存数据元素 $e$ 
```

```
    s->next=p->next;
```

```
    p->next=s; //将结点 $s$ 连到 $p$ 之后
```

```
    return true;
```

无其他结点

考虑到非法情况：健壮性

开”

按值查找

//按值查找, 找到数据域==e 的结点

```
ListNode * LocateElem(LinkList L, ElemType e) {
```

```
    → ListNode *p = L->next;
```

//从第1个结点开始查找数据域为e的结点

```
    → while (p != NULL && p->data != e)
```

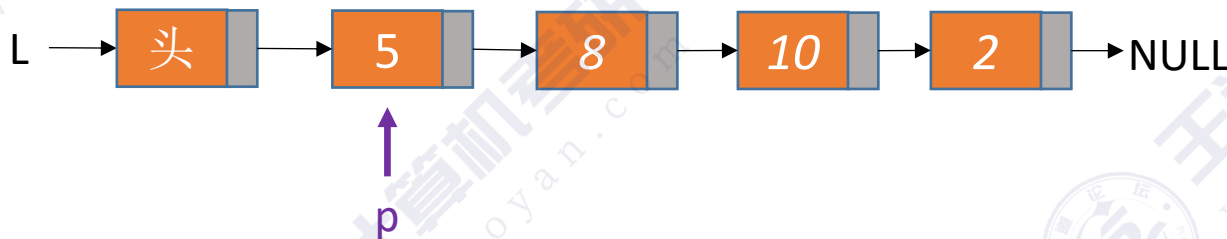
```
    → |     p = p->next;
```

```
    → return p; //找到后返回该结点指针, 否则返回NULL
}
```

注: 假设本例中
ElemType 是 int

能找到的情况:

①如果 $e = 8$



按值查找

//按值查找, 找到数据域==e 的结点

```
ListNode * LocateElem(LinkList L, ElemType e) {
```

```
    → ListNode *p = L->next;
```

```
    //从第1个结点开始查找数据域为e的结点
```

```
    → while (p != NULL && p->data != e)
```

```
    → |     p = p->next;
```

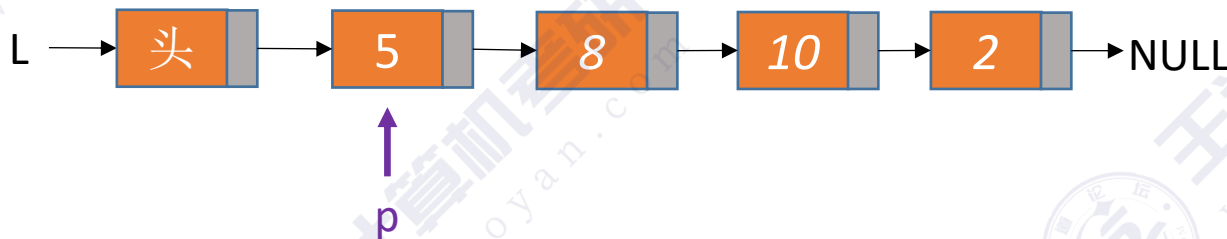
```
    → return p;    //找到后返回该结点指针, 否则返回NULL
}
```

注: 假设本例中
ElemType 是 int

不能找到的情况:

②如果 $e = 6$

如果 ElemType 是更复杂的结构类型呢?



平均时间复杂度: $O(n)$

求表的长度

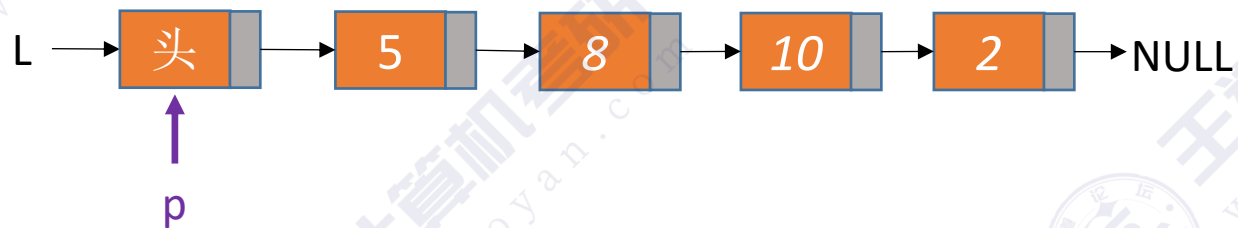
//求表的长度

```
int Length(LinkList L){  
    int len = 0;    //统计表长  
    LNode *p = L;  
    while (p->next != NULL){  
        p = p->next;  
        len++;  
    }  
    return len;  
}
```



如果不带头结点呢？

脑补一下咯



时间复杂度: $O(n)$

知识回顾与重要考点

单链表的查找

按位查找

注意与“顺序表”对比

单链表不具备“随机访问”的特性，只能依次扫描

按值查找

求单链表长度

三种基本操作的时间复杂度都是 $O(n)$

Key

如何写循环扫描各个结点的代码逻辑

注意边界条件的处理

本节内容

单链表 建立

知识总览

单链表的建立

尾插法

头插法



如果给你很多个数据元素（ElemType），要把它们存到一个单链表里边，咋 neng 呢？

Step 1: 初始化一个单链表

Step 2: 每次娶一个数据元素，插入到表尾/表头

本节探讨带头结点的情况

尾插法建立单链表

```
typedef struct LNode{           //定义单链表结点类型
    ElemType data;              //每个节点存放一个数据元素
    struct LNode *next;         //指针指向下一个节点
}LNode, *LinkList;

//初始化一个单链表（带头结点）
bool InitList(LinkList &L) {
    L = (LNode *) malloc(sizeof(LNode)); //分配一个头结点
    if (L==NULL)                          //内存不足，分配失败
        return false;
    L->next = NULL;                       //头结点之后暂时还没有节点
    return true;
}

void test(){
    LinkList L; //声明一个指向单链表的指针
    //初始化一个空表
    InitList(L);
    //.....后续代码.....
}
```



尾插法建立单链表

//在第 i 个位置插入元素 e (带头结点)

```
bool ListInsert(LinkList &L, int i, ElemType e){  
    if(i<1)  
        return false;  
    LNode *p;    //指针p指向当前扫描到的结点  
    int j=0;      //当前p指向的是第几个结点  
    p = L;        //L指向头结点, 头结点是第0个结点 (不存数据)  
    while (p!=NULL && j<i-1) { //循环找到第  $i-1$  个结点  
        p=p->next;  
        j++;  
    }  
    if(p==NULL) //i值不合法  
        return false;  
    LNode *s = (LNode *)malloc(sizeof(LNode));  
    s->data = e;  
    s->next=p->next;  
    p->next=s;    //将结点s连到p之后  
    return true;  //插入成功  
}
```

尾插法建立单链表:

初始化单链表

设置变量 $length$ 记录链表长度

While 循环 {

每次取一个数据元素 e ;

ListInsert (L, $length+1$, e) 插到尾部;

$length++$;

}

每次都从头开始之后遍历,
时间复杂度为 $O(n^2)$

垃圾



$length=3$

设置一个
表尾指针

↑
 r

尾插法建立单链表

//后插操作：在p结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){  
    if (p==NULL)  
        return false;  
    LNode *s = (LNode *)malloc(sizeof(LNode));  
    if (s==NULL) //内存分配失败  
        return false;  
    s->data = e; //用结点s保存数据元素e  
    s->next = p->next;  
    p->next = s; //将结点s连到p之后  
    return true;  
}
```

后插操作



设置一个
表尾指针

↑
r

尾插法建立单链表

```
LinkedList List_TailInsert(LinkedList &L){ //正向建立单链表
    int x; //设ElemType为整型
    L=(LinkedList)malloc(sizeof(LNode)); //建立头结点
    LNode *s,*r=L; //r为表尾指针
    scanf("%d",&x); //输入结点的值
    while(x!=9999){ //输入9999表示结束
        s=(LNode *)malloc(sizeof(LNode));
        s->data=x;
        r->next=s;
        r=s;
        scanf("%d",&x);
    }
    r->next=NULL; //尾结点指针置空
    return L;
}
```

初始化空表

在r结点之后插入元素 x

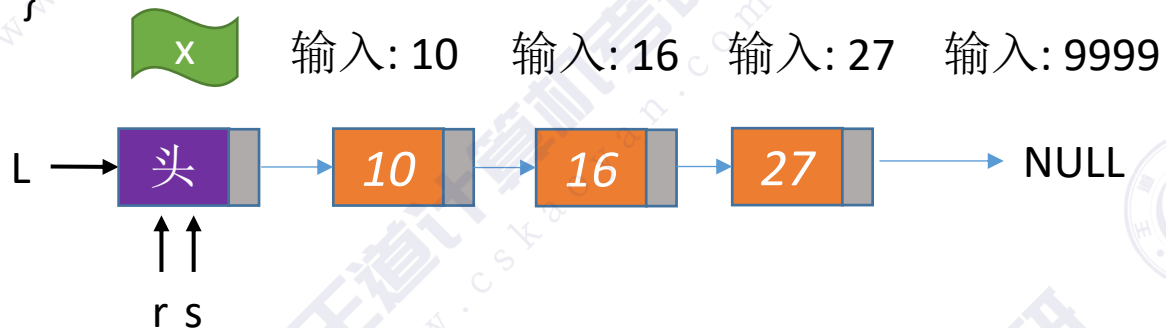
永远保持 r 指向
最后一个结点

时间复杂度: $O(n)$

如果不带头结点呢?

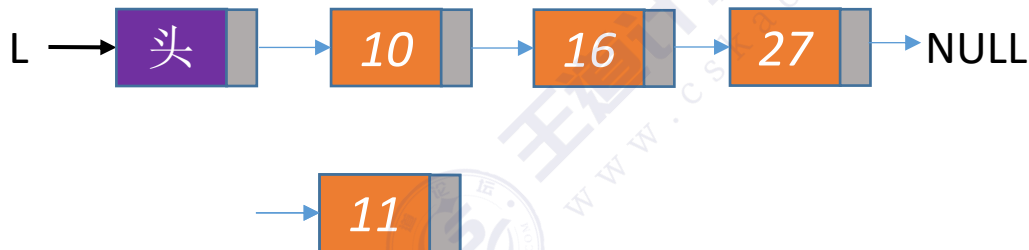


动口不动手



头插法建立单链表

对头结点的
后插操作



//后插操作：在 p 结点之后插入元素 e

```
bool InsertNextNode (LNode *p, ElemType e){  
    if (p==NULL)  
        return false;  
    LNode *s = (LNode *)malloc(sizeof(LNode));  
    if (s==NULL) //内存分配失败  
        return false;  
    s->data = e; //用结点s保存数据元素e  
    s->next=p->next;  
    p->next=s; //将结点s连到p之后  
    return true;  
}
```

头插法建立单链表：

初始化单链表

While 循环 {

 每次取一个数据元素 e ;
 InsertNextNode (L , e);

}

头插法建立单链表

养成好习惯，只要是初始化单链表，都先把头指针指向 NULL

```
LinkedList List_HeadInsert(LinkedList &L){ //逆向建立单链表
    LNode *s;
    int x;
    L=(LinkedList)malloc(sizeof(LNode)); //创建头结点
    L->next=NULL; //初始为空链表
    scanf("%d",&x); //输入结点的值
    while(x!=9999){ //输入9999表示结束
        s=(LNode*)malloc(sizeof(LNode)); //创建新结点
        s->data=x;
        s->next=L->next;
        L->next=s; //将新结点插入表中，L为头指针
        scanf("%d",&x);
    }
    return L;
}
```

如果去掉这一句呢？

```
//后插操作：在p结点之后插入元素 e
bool InsertNextNode (LNode *p, ElemType e){
    if (p==NULL)
        return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s==NULL) //内存分配失败
        return false;
    s->data = e; //用结点s保存数据元素e
    s->next=p->next;
    p->next=s; //将结点s连到p之后
    return true;
}
```

重要应用!!!
链表的逆置



如果不带头结点呢？

一定要动手鸭



各位，多说无益，动手吧.....

知识回顾与重要考点

头插法、尾插法：核心就是初始化操作、指定结点的后插操作

注意设置一个指向
表尾结点的指针



头插法的重要应用：链表的逆置

动手试一试：给你一个 LinkList L，
如何逆置？



本节内容

双链表

知识总览

双链表

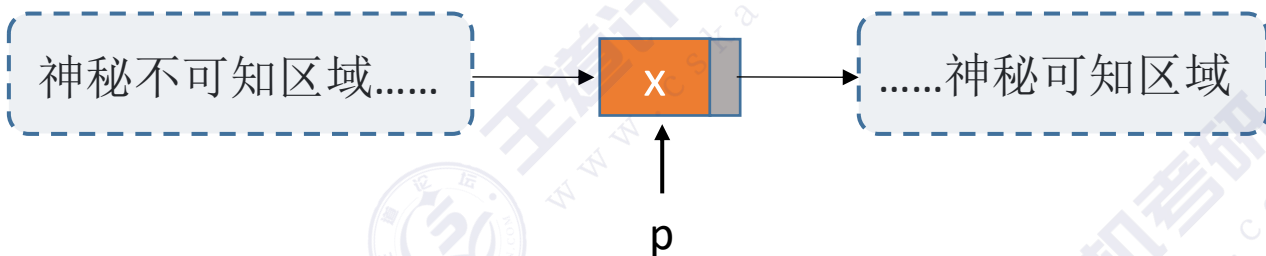
初始化

插入

删除

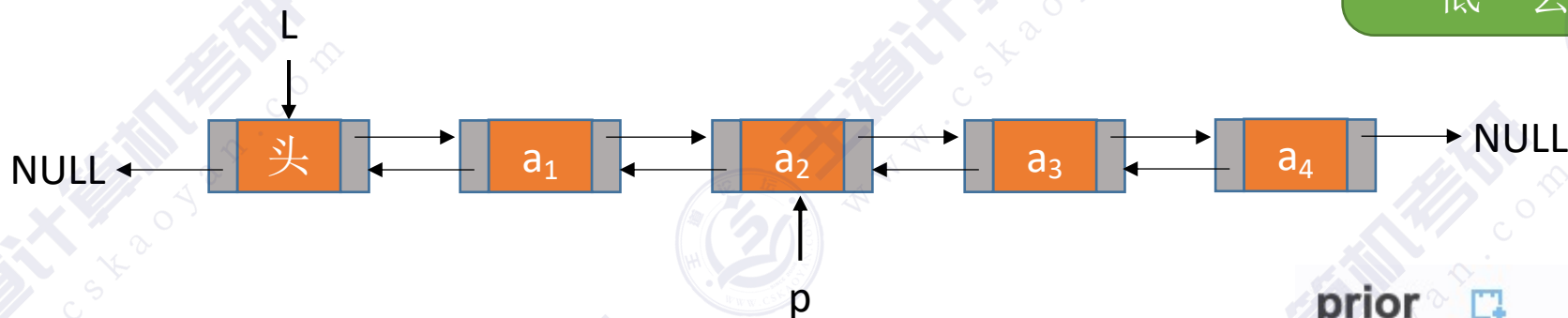
遍历

单链表 V.S. 双链表



单链表：无法逆向检索，有时候不太方便

双链表：可进可退，存储密度更低一丢丢



```
typedef struct DNode{  
    ElemType data;  
    struct DNode *prior, *next;  
}DNode, *DLinklist;
```

//定义双链表结点类型
//数据域
//前驱和后继指针

prior

英 ['praɪə(r)]

美 ['praɪər]

adj. (时间、顺序等) 先前的; 优先的

n. 男修道院副院长; 托钵会会长; (非正式) 犯罪前科; 先验;

n. (Prior) (美) 普廖尔 (人名)

[复数 priors]

双链表的初始化（带头结点）

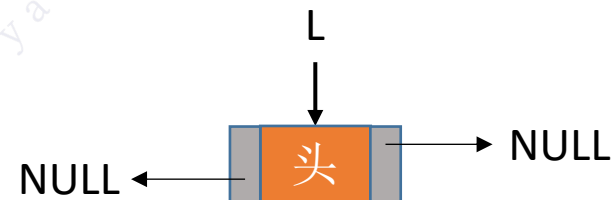
//初始化双链表

```
bool InitDLinklist(DLinklist &L){  
    L = (DNode *) malloc(sizeof(DNode));    //分配一个头结点  
    if (L==NULL)    //内存不足, 分配失败  
        return false;  
    L->prior = NULL;    //头结点的 prior 永远指向 NULL  
    L->next = NULL;    //头结点之后暂时还没有节点  
    return true;  
}
```

```
void testDLinkList() {  
    //初始化双链表  
    DLinklist L;  
    InitDLinkList(L);  
    //后续代码...  
}  
  
//判断双链表是否为空（带头结点）  
bool Empty(DLinklist L) {  
    if (L->next == NULL)  
        return true;  
    else  
        return false;  
}
```

```
typedef struct DNode{  
    ElemType data;  
    struct DNode *prior,*next;  
}DNode, *DLinklist;
```

DLinklist $\longleftrightarrow$ 等价 $\longleftrightarrow$ DNode *



双链表的插入

//在p结点之后插入s结点

```
bool InsertNextDNode(DNode *p, DNode *s){
```

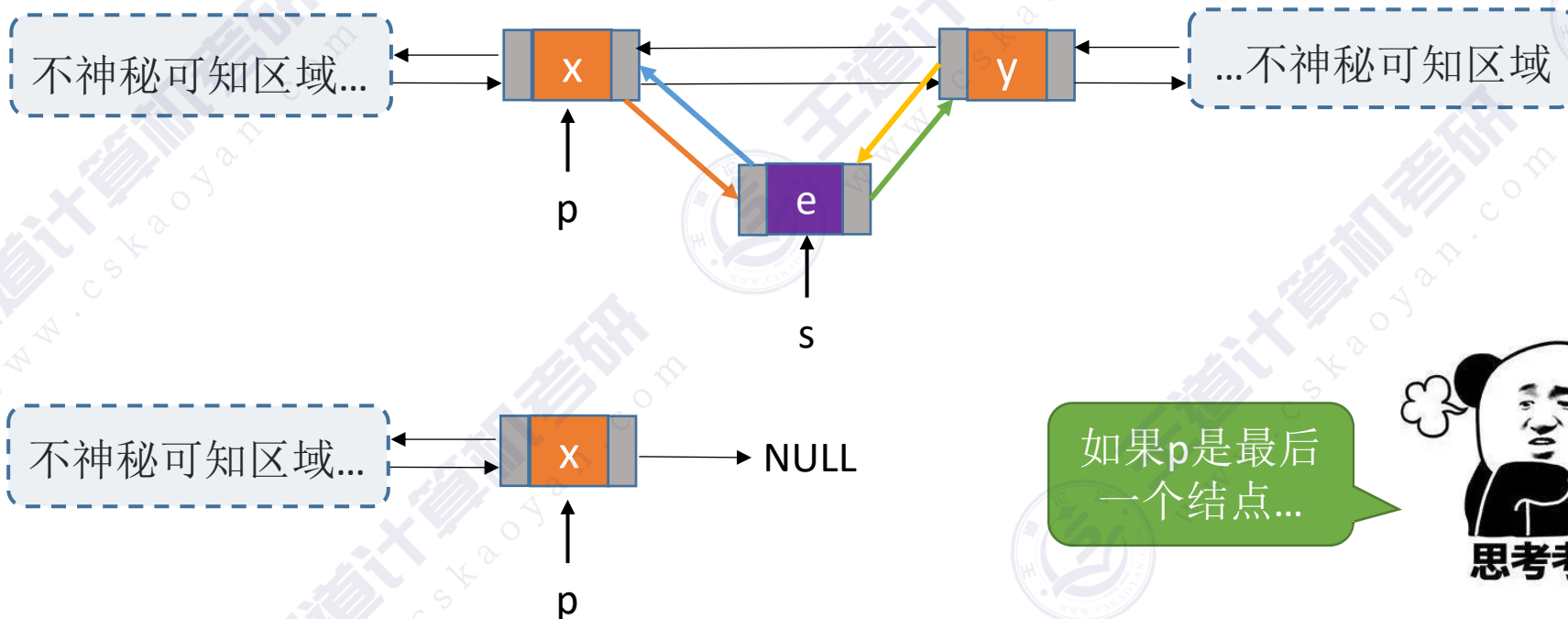
```
    s->next=p->next;    //将结点*s插入到结点*p之后
```

```
    p->next->prior=s;
```

```
    s->prior=p;
```

```
    p->next=s;
```

```
}
```



如果p是最后
一个结点...



思考考

双链表的插入

//在p结点之后插入s结点

```
bool InsertNextDNode(DNode *p, DNode *s){  
    if (p==NULL || s==NULL) //非法参数  
        return false;  
    ① s->next=p->next;  
    ② if (p->next != NULL) //如果p结点有后继结点  
        p->next->prior=s;  
    ③ s->prior=p;  
    ④ p->next=s;  
    return true;  
}
```

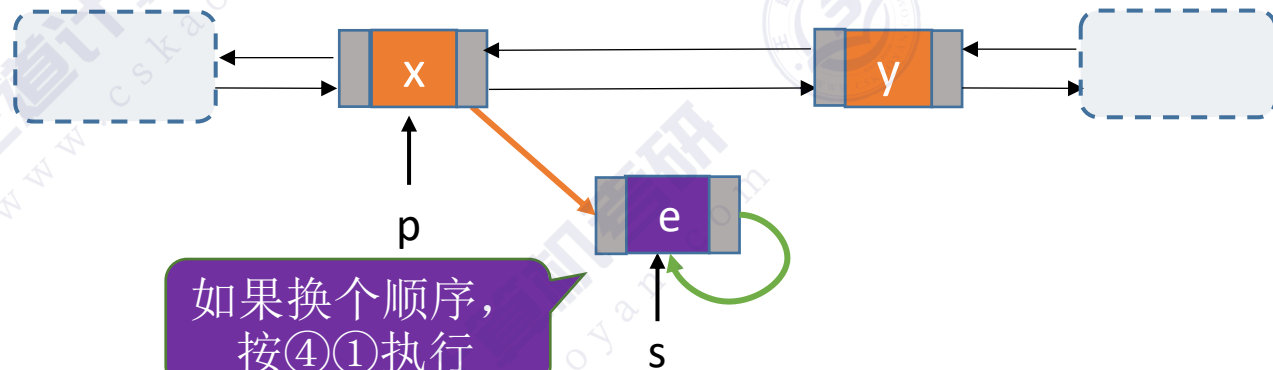
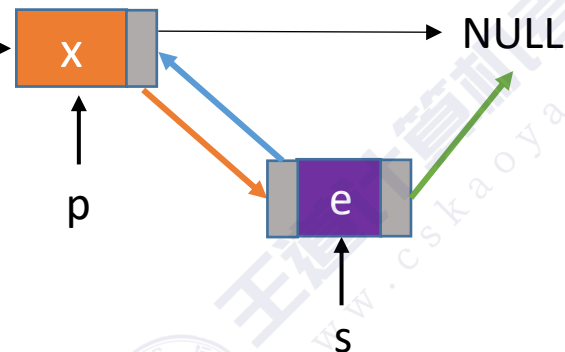
修改指针时要注意顺序

用后插操作实现结点的插入有什么好处？

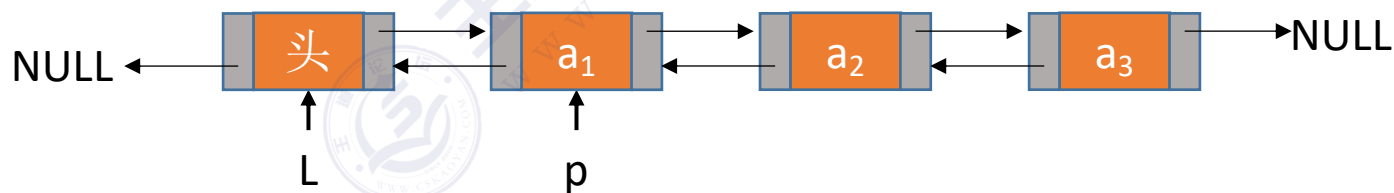
按位序插入
前插操作



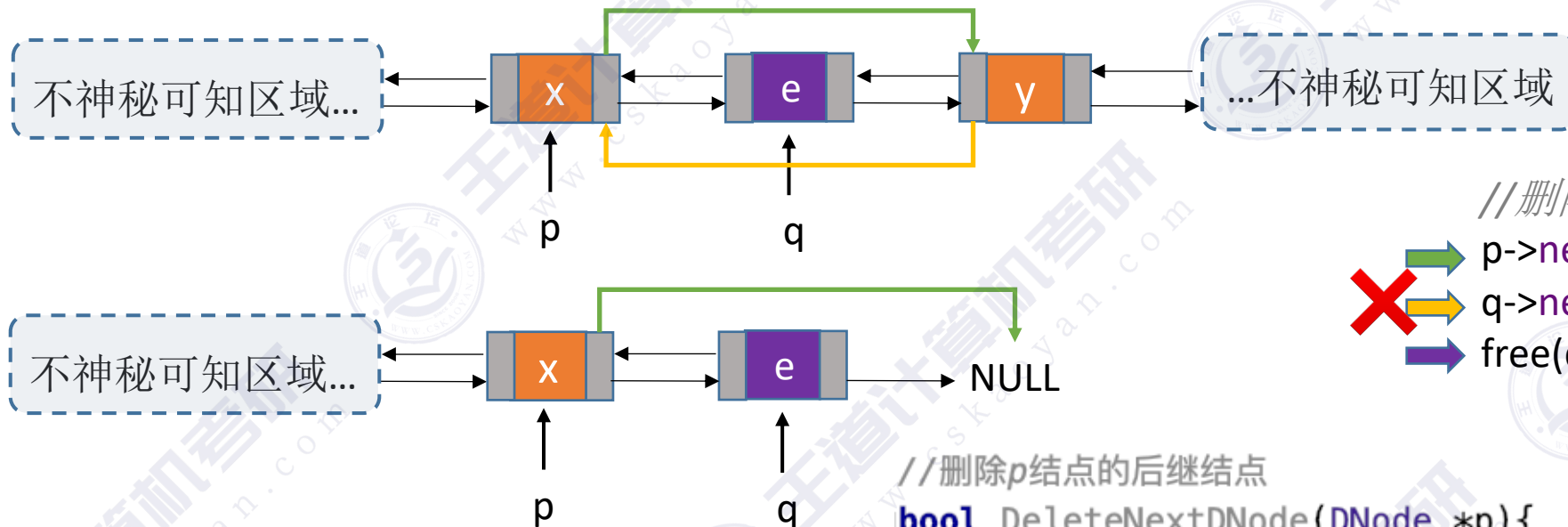
神秘未知区域.....



如果换个顺序,
按④①执行



双链表的删除



//删除p的后继结点q

→ p->next=q->next;
✗ → q->next->prior=p;
→ free(q);

//删除p结点的后继结点

```
bool DeleteNextDNode(DNode *p){  
    if (p==NULL) return false;  
    DNode *q = p->next; //找到p的后继结点q  
    if (q==NULL) return false; //p没有后继  
    → p->next=q->next;  
    → if (q->next!=NULL) //q结点不是最后一个结点  
        q->next->prior=p;  
    → free(q); //释放结点空间  
    return true;  
}
```

后删操作实现结点的删除有什么好处?

销毁表时才能删除头结点

```
void DestroyList(DLinklist &L){  
    //循环释放各个数据结点  
    while (L->next != NULL)  
        DeleteNextDNode(L);  
    free(L); //释放头结点  
    L=NULL; //头指针指向NULL  
}
```

双链表的遍历

后向遍历

```
while (p!=NULL){  
    //对结点p做相应处理，如打印  
    p = p->prior;  
}
```

前向遍历

```
while (p!=NULL){  
    //对结点p做相应处理  
    p = p->next;  
}
```

前向遍历（跳过头结点）

```
while (p->prior != NULL){  
    //对结点p做相应处理  
    p = p->prior;  
}
```

双链表不可随机存取，按位查找、按值查找操作都只能用遍历的方式实现。时间复杂度 $O(n)$

知识回顾与重要考点

双链表

如果不带头
结点呢？

初始化

头结点的 prior、next 都指向 NULL

插入 (后插)

注意新插入结点、前驱结点、后继结点的指针修改

边界情况：新插入结点在最后一个位置，需特殊处理

删除 (后删)

注意删除结点的前驱结点、后继结点的指针修改

边界情况：如果被删除结点是最后一个数据结点，需特殊处理

遍历

从一个给定结点开始，后向遍历、前向遍历的实现（循环的终止条件）

链表不具备随机存取特性，查找操作只能通过顺序遍历实现

本节内容

循环链表

知识总览

循环链表

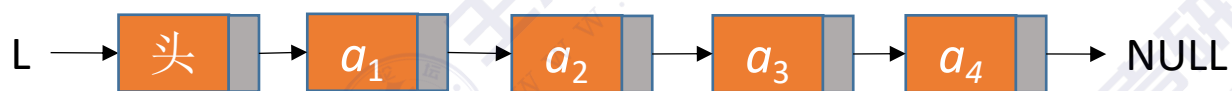
循环单链表

循环双链表

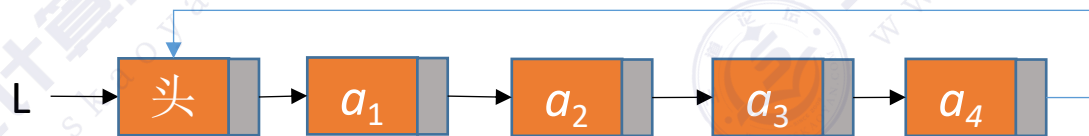
就那么简单



循环单链表



单链表：表尾结点的next指针指向 NULL



循环单链表：表尾结点的next指针指向头结点

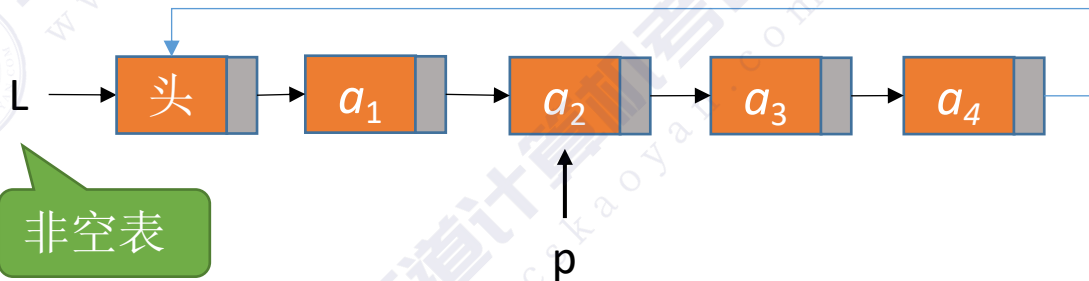
循环单链表

```
typedef struct LNode{  
    ElemType data;  
    struct LNode *next;  
}LNode, *LinkList;
```

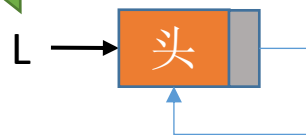
//定义单链表结点类型
//每个节点存放一个数据元素
//指针指向下一个节点

//初始化一个循环单链表

```
bool InitList(LinkList &L) {  
    L = (LNode *) malloc(sizeof(LNode)); //分配一个头结点  
    if (L==NULL) //内存不足, 分配失败  
        return false;  
    L->next = L; //头结点next指向头结点  
    return true;  
}
```



空表



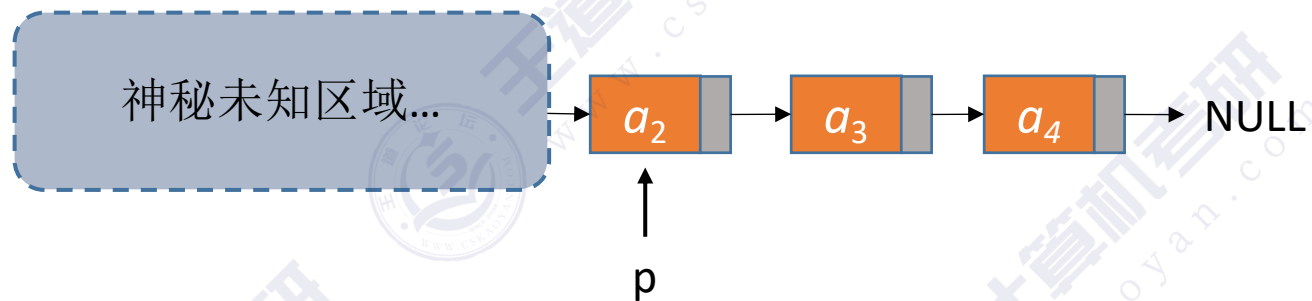
//判断循环单链表是否为空

```
bool Empty(LinkList L) {  
    if (L->next == L)  
        return true;  
    else  
        return false;  
}
```

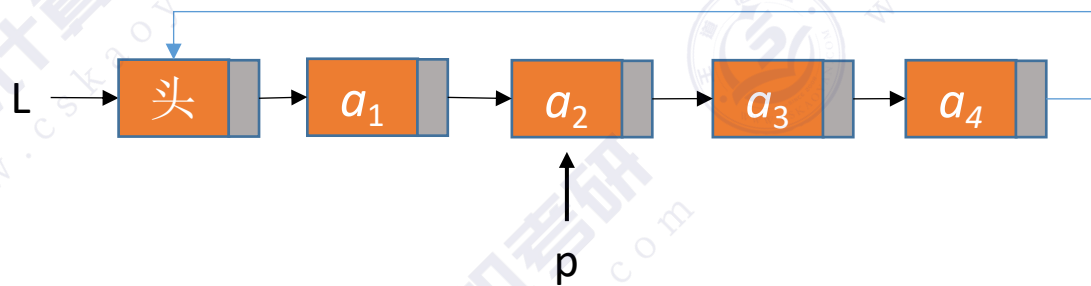
//判断结点p是否为循环单链表的表尾结点

```
bool isTail(LinkList L, LNode *p){  
    if (p->next==L)  
        return true;  
    else  
        return false;  
}
```

循环单链表

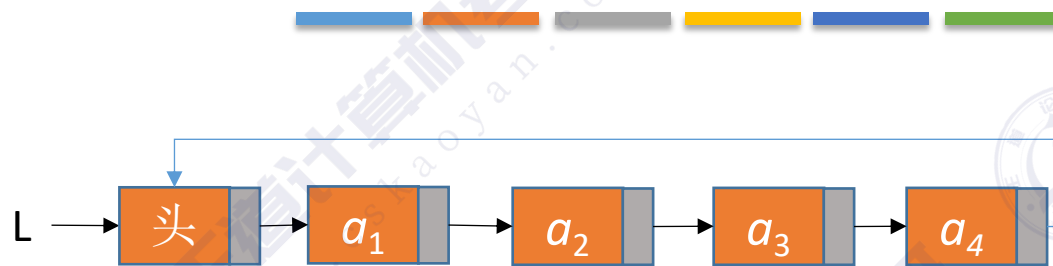


单链表：从一个结点出发只能找到后续的各个结点



循环单链表：从一个结点出发可以找到其他任何一个结点

循环单链表



从头结点找到尾部，
时间复杂度为 $O(n)$

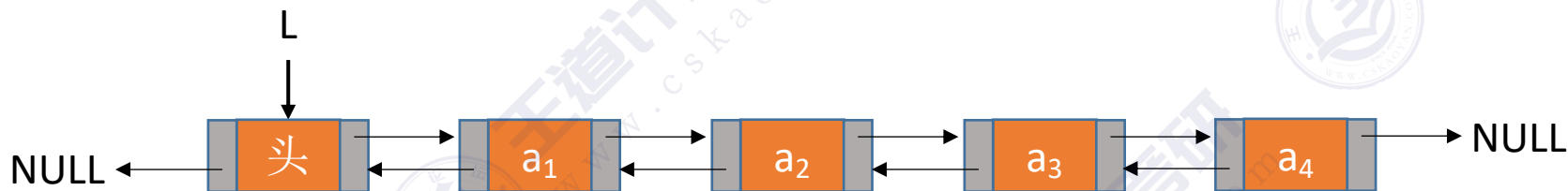
从尾部找到头部，
时间复杂度为 $O(1)$



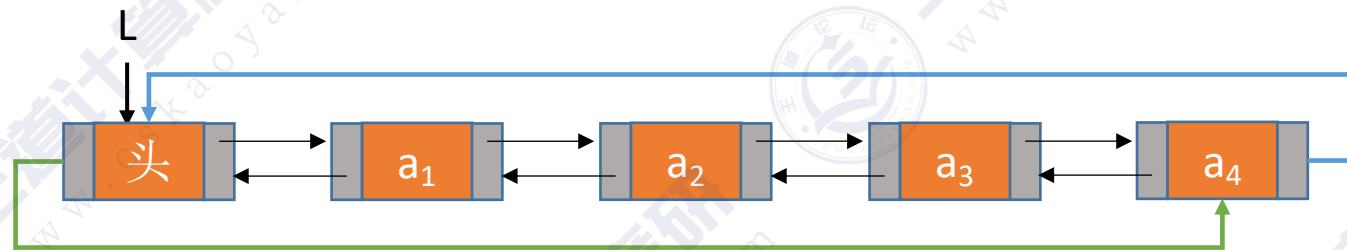
很多时候对链表的操作
都是在头部或尾部

可以让L指向表尾元素
(插入、删除时可能需要修改L)

循环双链表



双链表：
表头结点的 prior 指向 NULL；
表尾结点的 next 指向 NULL



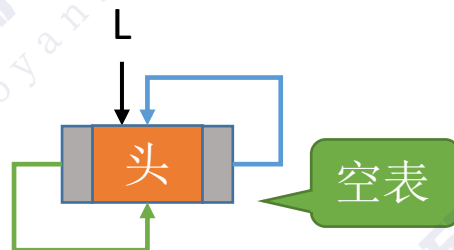
循环双链表：
表头结点的 prior 指向表尾结点；
表尾结点的 next 指向头结点

循环双链表的初始化

//初始化空的循环双链表

```
bool InitDLinkedList(DLinkedList &L){
    L = (DNode *) malloc(sizeof(DNode));    //分配一个头结点
    if (L==NULL)                            //内存不足, 分配失败
        return false;
    L->prior = L;    //头结点的 prior 指向头结点
    L->next = L;    //头结点的 next 指向头结点
    return true;
}
```

```
typedef struct DNode{
    ElemType data;
    struct DNode *prior,*next;
}DNode, *DLinkedList;
```



```
void testDLinkedList() {
    //初始化循环双链表
    DLinkedList L;
    InitDLinkedList(L);
    //...后续代码...
}
```

//判断循环双链表是否为空

```
bool Empty(DLinkedList L) {
    if (L->next == L)
        return true;
    else
        return false;
}
```

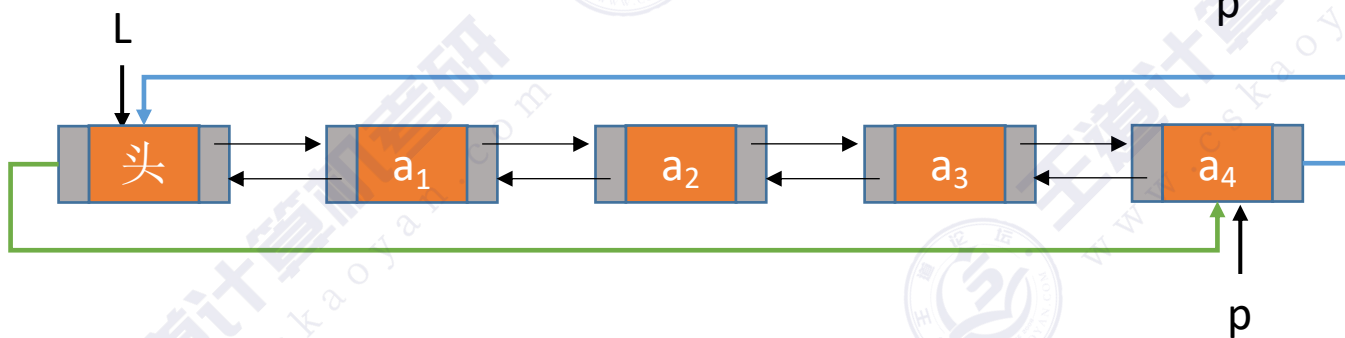
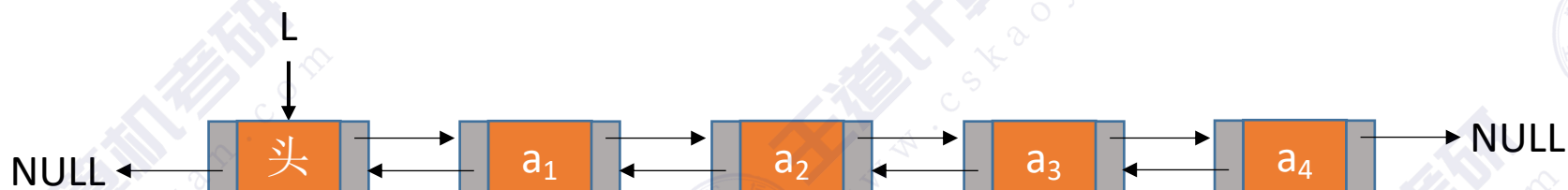
//判断结点p是否为循环单链表的表尾结点

```
bool isTail(DLinkedList L, DNode *p){
    if (p->next==L)
        return true;
    else
        return false;
}
```

双链表的插入

//在p结点之后插入s结点

```
bool InsertNextDNode(DNode *p, DNode *s){  
    s->next=p->next;    //将结点*s插入到结点*p之后  
    p->next->prior=s;  
    s->prior=p;  
    p->next=s;  
}
```



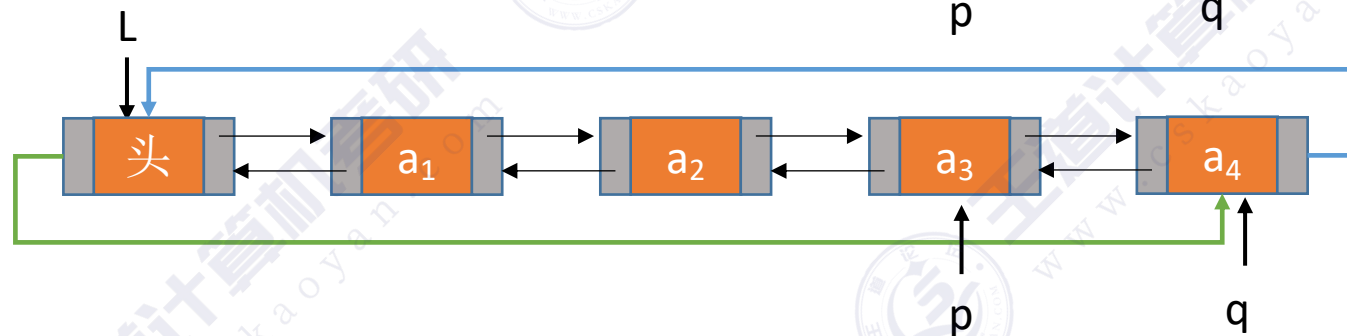
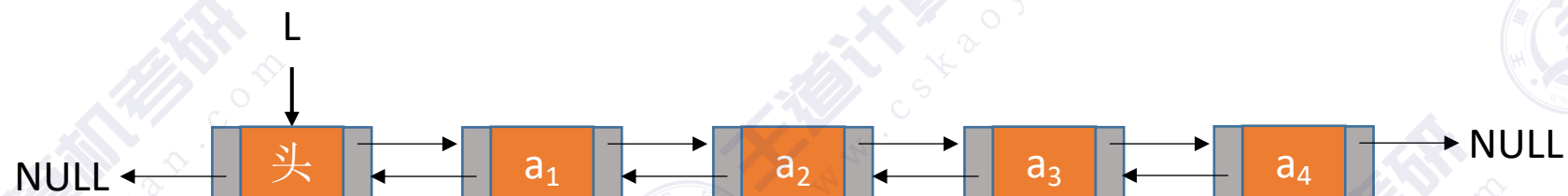
双链表的删除

//删除 p 的后继结点 q

$p \rightarrow \text{next} = q \rightarrow \text{next};$

→ $q \rightarrow \text{next} \rightarrow \text{prior} = p;$

$\text{free}(q);$

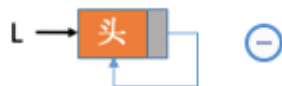


知识回顾与重要考点

循环链表

循环单链表

空表



单手抱住空虚的自己



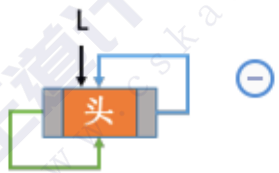
非空表



普通的空虚双链表

循环双链表

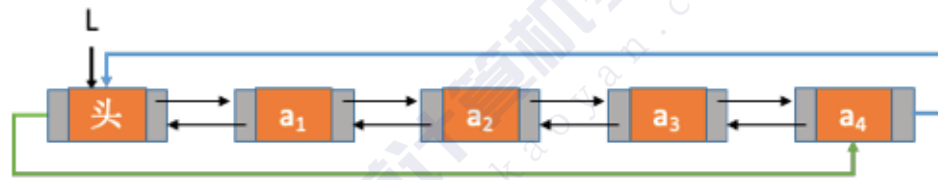
空表



双手抱住空虚的自己



非空表



心疼却抱不住这么胖的自己



如何判空

代码问题

如何判断结点p是否是表尾/表头结点

后向/前向遍历的实现核心

如何在表头、表中、表尾插入/删除一个结点

插入、删除操作的不易错思路

本节内容

静态链表

知识总览

静态链表

什么是静态链表

如何定义一个静态链表

简述基本操作的实现

什么是静态链表

静态链表

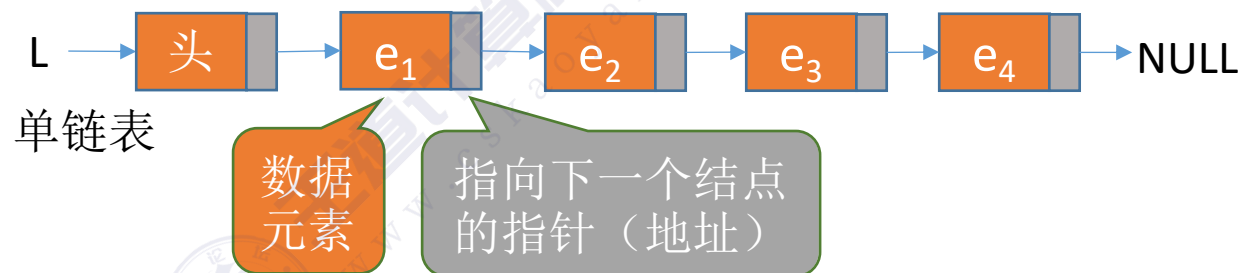


单链表：各个结点在内存中星罗棋布、散落天涯。

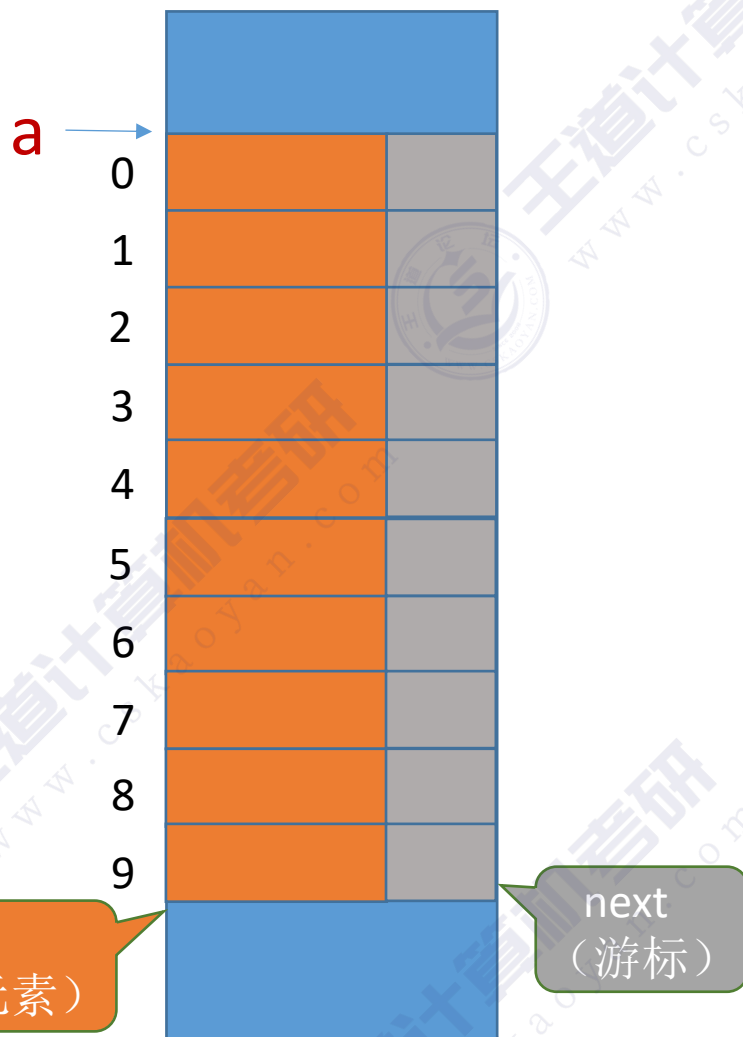
静态链表：分配一整片连续的内存空间，各个结点集中安置。

每个数据元素 4B，每个游标 4B（每个结点共 8B）
设起始地址为 **addr**

e₁ 的存放地址为 **addr + 8*2**



用代码定义一个静态链表



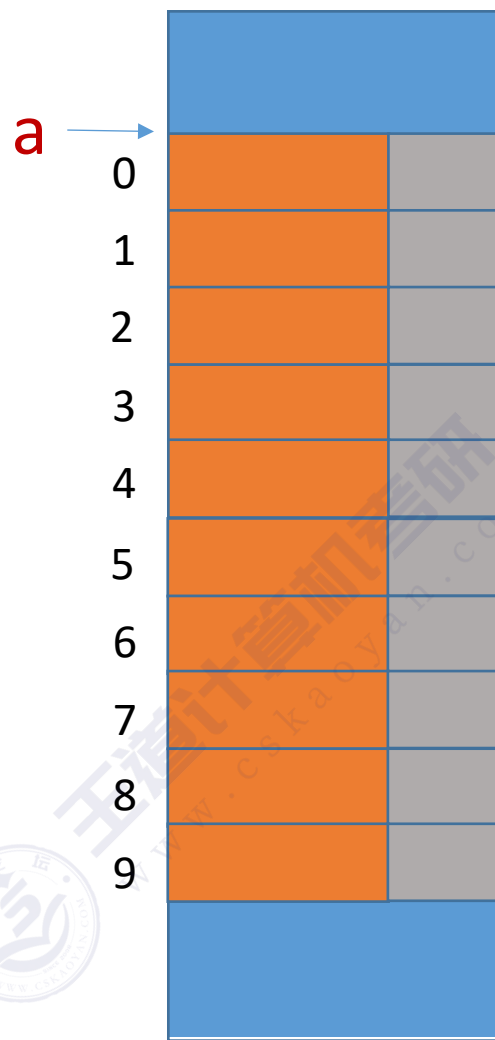
```
#define MaxSize 10  
struct Node{  
    ElemType data;  
    int next;  
};
```

//静态链表的最大长度
//静态链表结构类型的定义
//存储数据元素
//下一个元素的数组下标

```
void testSLinkList() {  
    struct Node a[MaxSize];  
    //.....后续代码  
}
```

数组 a 作为
静态链表

用代码定义一个静态链表



内存

```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;
} SLinkList[MaxSize];
```

//静态链表的最大长度
//静态链表结构类型的定义
//存储数据元素
//下一个元素的数组下标



```
#define MaxSize 10
struct Node{
    ElemType data;
    int next;
};
typedef struct Node SLinkList[MaxSize];
```

可用 SLinkList 定义 “一个长度为 MaxSize 的Node 型数组”

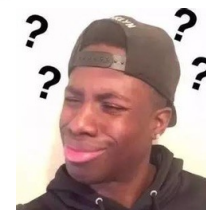
a 是一个静态链表

```
void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```



```
void testSLinkList() {
    struct Node a[MaxSize];
    //.....后续代码
}
```

a 是一个Node型数组



对猜想的验证

```
#define MaxSize 10 //静态链表的最大长度
struct Node{       //静态链表结构类型的定义
    int data;       //存储数据元素
    int next;       //下一个元素的数组下标
};
typedef struct {    //静态链表结构类型的定义
    int data;       //存储数据元素
    int next;       //下一个元素的数组下标
} SLinkList[MaxSize];

void testSLinkList() {
    struct Node x;
    printf("sizeX=%d\n", sizeof(x));

    struct Node a[MaxSize];
    printf("sizeA=%d\n", sizeof(a));

    SLinkList b;
    printf("sizeB=%d\n", sizeof(b));
}
```

结论:

SLinkList b —— 相当于定义了一个长度为 MaxSize 的 Node 型数组

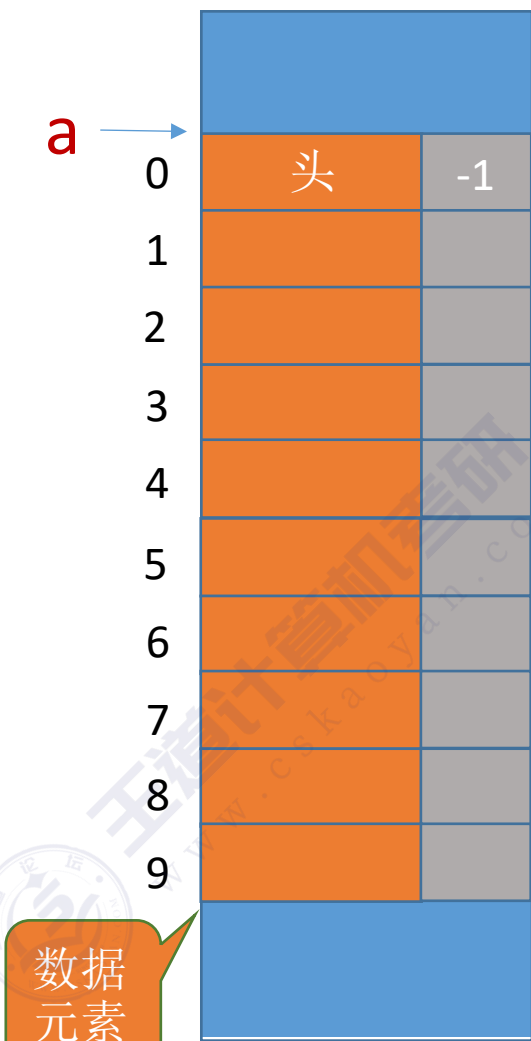
运行结果

```
sizeX=8
sizeA=80
sizeB=80
```

Process finished with exit code 0

简述基本操作的实现

静态链表



```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;
} SLinkList[MaxSize];
```

//静态链表的最大长度
//静态链表结构类型的定义
//存储数据元素
//下一个元素的数组下标

```
void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```

初始化静态链表:

把 a[0] 的 next 设为 -1

把其他结点的 next 设为一个特殊
值用来表示结点空闲, 如 -2



简述基本操作的实现

静态链表

a →

0	头	2
1	e ₂	6
2	e ₁	1
3	e ₄	-1
4		
5		
6	e ₃	3
7		
8		
9		

数据元素

内存

```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;
} SLinkList[MaxSize];
```

//静态链表的最大长度
//静态链表结构类型的定义
//存储数据元素
//下一个元素的数组下标

```
void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```

查找:
从头结点出发挨个往后遍历结点

插入位序为 i 的结点:

- ① 找到一个空的结点, 存入数据元素
- ② 从头结点出发找到位序为 i-1 的结点
- ③ 修改新结点的 next
- ④ 修改 i-1 号结点的 next

如何判断结点是否为空?

可让 next 为某个特殊值, 如 -2

删除某个结点:

- ① 从头结点出发找到前驱结点
- ② 修改前驱结点的游标
- ③ 被删除结点 next 设为 -2

知识回顾与重要考点

静态链表

a →

0	头	2
1	e ₂	6
2	e ₁	1
3	e ₄	-1
4		-2
5		-2
6	e ₃	3
7		-2
8		-2
9		-2

数据元素

0号结点充当“头结点”

游标为 -1 表示已经到达表尾

游标充当“指针”

用一个特殊的数值标记空闲结点

下一个节点的数组下标 (游标)

```
#define MaxSize 10
typedef struct {
    ElemType data;
    int next;
} SLinkList[MaxSize];

void testSLinkList() {
    SLinkList a;
    //.....后续代码
}
```

//静态链表的最大长度
//静态链表结构类型的定义
//存储数据元素
//下一个元素的数组下标

静态链表：用数组的方式实现的链表

优点：增、删 操作不需要大量移动元素

缺点：不能随机存取，只能从头结点开始依次往后查找；容量固定不可变

适用场景：①不支持指针的低级语言；②数据元素数量固定不变的场景（如操作系统的文件分配表FAT）

本节内容

顺序表 V.S. 链表

知识总览

顺序表
V.S.
链表

Round 1



逻辑结构

Round 2



物理结构/存储结构

Round 3

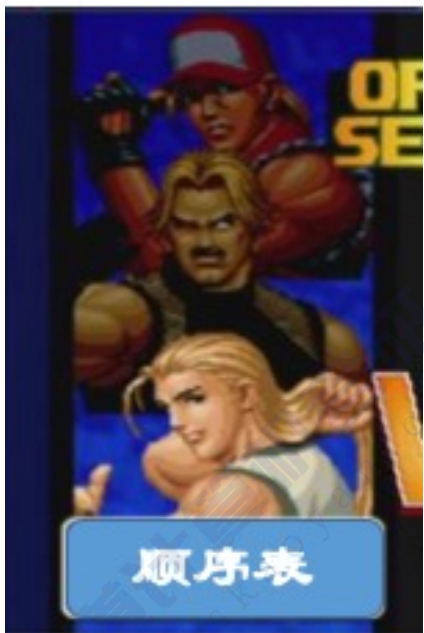


数据的运算/基本操作

如何抉择? ❤️到底爱谁? 💖



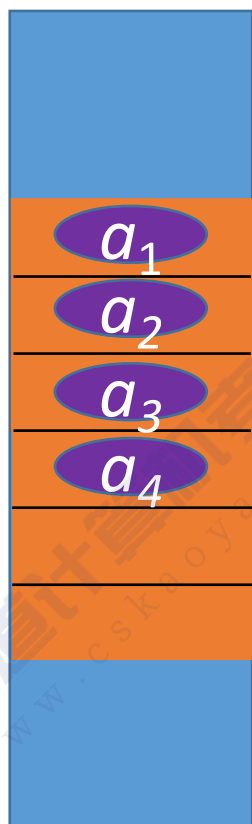
Round 1: 逻辑结构



都属于线性表，都是线性结构

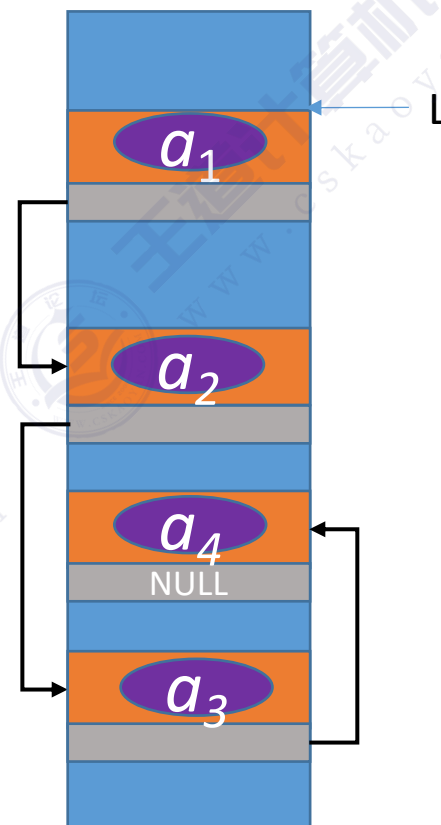
Round 2: 存储结构

顺序表
(顺序存储)



都属于线性表，都是线性结构

链表
(链式存储)



优点：支持随机存取、存储密度高
缺点：大片连续空间分配不方便，改变容量不方便

优点：离散的小空间分配方便，改变容量方便
缺点：不可随机存取，存储密度低

Round 3: 基本操作



复习回忆思路:

创销、增删改查

Round 3: 基本操作

创

顺序表
(顺序存储)



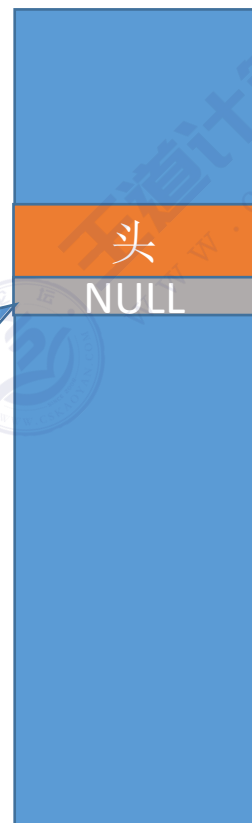
需要预分配大片连续空间。
若分配空间过小，则之后不方便拓展容量；若分配空间过大，则浪费内存资源

静态分配：静态数组
动态分配：动态数组 (malloc、free)

容量不可改变

容量可改变，但需要移动大量元素，时间代价高

链表
(链式存储)



只需分配一个头结点（也可以不要头结点，只声明一个头指针），之后方便拓展

Round 3: 基本操作

销

顺序表
(顺序存储)



修改 Length = 0

L

静态分配: 静态数组
动态分配: 动态数组 (malloc、free)

系统自动回收空间

需要手动 free

```
typedef struct{
    ElemType *data;
    int MaxSize;
    int length;
} SeqList;
```

malloc 和 free
必须成对出现

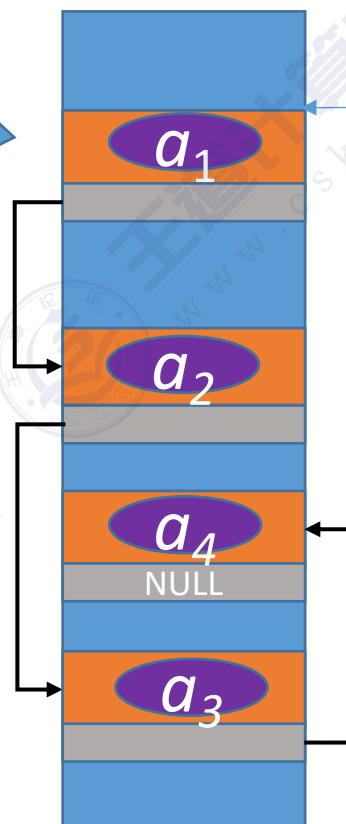
L.data = (ElemType *) malloc(sizeof(ElemType) * InitSize);

free(L.data);

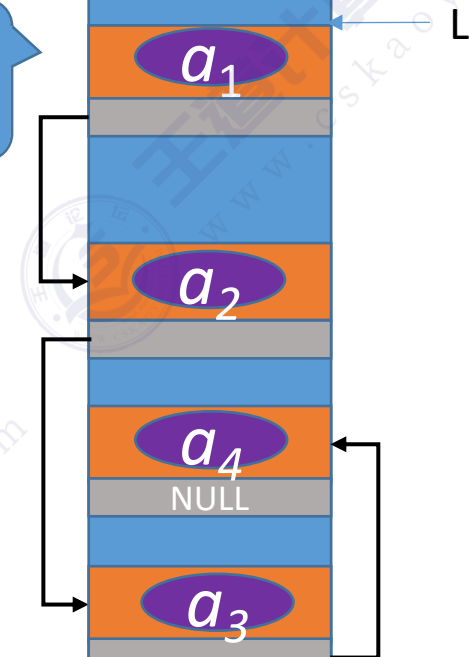
销

创

链表
(链式存储)



依次删除各个
结点 (free)

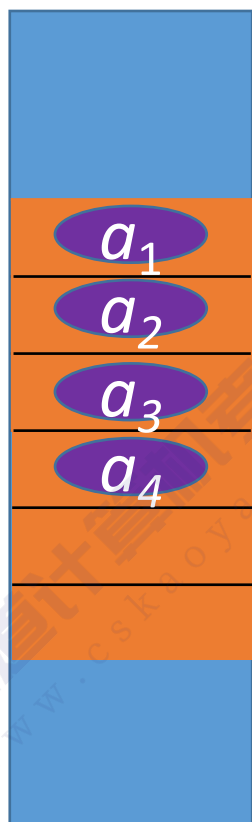


Round 3: 基本操作

增

删

顺序表
(顺序存储)



插入/删除元素要将后续元素都后移/前移

时间复杂度 $O(n)$ ，时间开销主要来自移动元素

若数据元素很大，则移动的时间代价很高

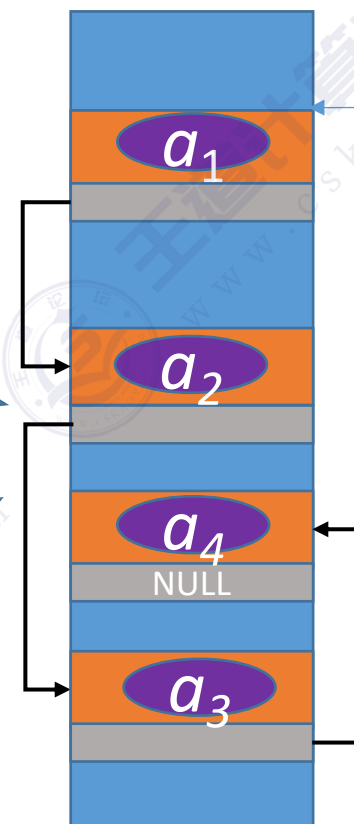


插入/删除元素只需修改指针即可

时间复杂度 $O(n)$ ，时间开销主要来自查找目标元素

查找元素的时间代价更低

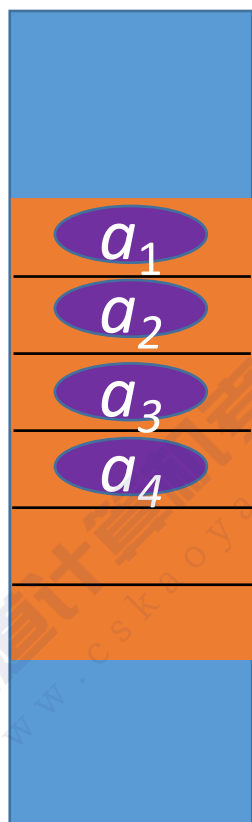
链表
(链式存储)



Round 3: 基本操作

查

顺序表
(顺序存储)

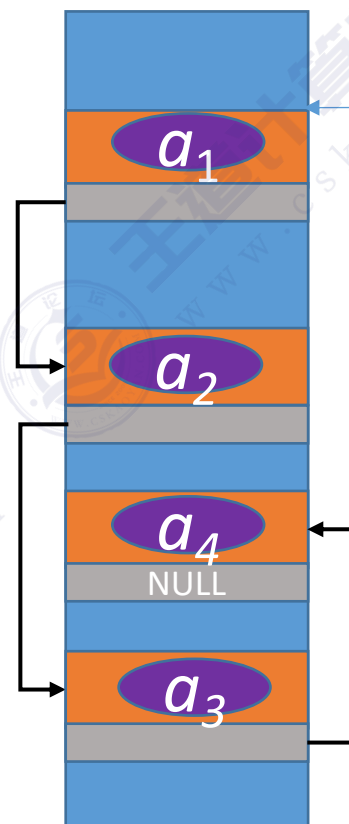


按位查找: $O(1)$

按值查找: $O(n)$
若表内元素有序, 可在
 $O(\log_2 n)$ 时间内找到



链表
(链式存储)



按位查找: $O(n)$

按值查找: $O(n)$

用顺序表 or 链表?

	顺序表	链表
弹性（可扩容）	😭	😄
增、删	😭	😄
查	😄	😭

表长难以预估、经常要增加/删除元素

——链表

表长可预估、查询（搜索）操作较多

——顺序表



知识回顾与重要考点



开放式问题的答题思路:

问题： 请描述顺序表和链表的 bla bla bla...
实现线性表时，用顺序表还是链表好？

顺序表和链表的**逻辑结构**都是线性结构，都属于线性表。

但是二者的**存储结构**不同，顺序表采用顺序存储...(特点，带来的优点缺点)；链表采用链式存储...(特点、导致的优缺点)。

由于采用不同的存储方式实现，因此**基本操作**的实现效率也不同。当初始化时...；当插入一个数据元素时...；当删除一个数据元素时...；当查找一个数据元素时...